2026_top_10_llm_update_what_the_data_says

May 26, 2026

[1]:
```
# --- Act 0: Setup (hidden) ---
import warnings
warnings.filterwarnings('ignore')

import html
import json
import re
from pathlib import Path
from collections import Counter, defaultdict
from textwrap import shorten

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.ticker as ticker
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
import plotly.io as pio
from scipy import stats
from IPython.display import HTML, display, Markdown

# --- Path resolution ---
_here = Path('.').resolve()
if (_here / 'projects').exists():
    REPO_ROOT = _here
elif (_here.parent / 'projects').exists():
    REPO_ROOT = _here.parent
else:
    raise FileNotFoundError(
        "Cannot find 'projects/' directory. "
        "Run this notebook from the repository root or the notebooks/ directory.
 ↪"
    )
CYCLE = REPO_ROOT / 'projects' / 'owasp-llm' / 'cycles' / '2026'
```

```
# --- Plotly config ---
# "notebook" renderer produces interactive HTML via plotly.js.
# We omit "+png" because kaleido (the static image engine) is blocked on
# ARM/Tegra to prevent a SIGABRT in tensorflow/jaxlib native code.
pio.renderers.default = "notebook"
pio.templates.default = "plotly_white"

# --- Seaborn / matplotlib theme ---
sns.set_theme(style='whitegrid', font_scale=1.1)
plt.rcParams.update({
    'figure.figsize': (12, 6),
    'font.size': 12,
    'axes.titlesize': 14,
    'axes.labelsize': 12,
    'figure.dpi': 150,
})

# --- Color palette ---
ENTRY_IDS = [
    'LLM01', 'LLM02', 'LLM03', 'LLM04', 'LLM05',
    'LLM06', 'LLM07', 'LLM08', 'LLM09', 'LLM10',
    'NEW-ITSCD', 'NEW-MA', 'NEW-MSDA', 'NEW-MTIE', 'NEW-PMP', 'NEW-WLA',
    'ROLL-CFAS', 'ROLL-CMSB', 'ROLL-LAPTF', 'ROLL-SICG',
]
FRAME_BLIND = {'LLM04', 'LLM08', 'LLM10'}

_incumbent_blues = sns.color_palette('mako', n_colors=12)
_new_oranges = sns.color_palette('flare', n_colors=8)
_roll_purples = sns.color_palette('crest', n_colors=6)
ENTRY_COLORS = {}
_ib = 0; _in = 0; _ir = 0
for eid in ENTRY_IDS:
    if eid in FRAME_BLIND:
        ENTRY_COLORS[eid] = '#999999'
    elif eid.startswith('LLM'):
        ENTRY_COLORS[eid] = _incumbent_blues[_ib % len(_incumbent_blues)]
        _ib += 1
    elif eid.startswith('NEW'):
        ENTRY_COLORS[eid] = _new_oranges[_in % len(_new_oranges)]
        _in += 1
    elif eid.startswith('ROLL'):
        ENTRY_COLORS[eid] = _roll_purples[_ir % len(_roll_purples)]
        _ir += 1

# --- Deep-dive sidebar helper ---
def sidebar(title, content):
    """Render a collapsible deep-dive sidebar."""
```

```
    return HTML(
        f'<details style="margin: 1em 0; padding: 0.5em; '
        f'border-left: 3px solid #4a86c8; background: #f8f9fa;">'
        f'<summary style="cursor: pointer; font-weight: bold; '
        f'color: #2c5282;">{title}</summary>'
        f'<div style="margin-top: 0.8em; line-height: 1.6;">{content}</div>'
        f'</details>'
    )

def callout_box(text, border_color='#e53e3e', bg_color='#fff5f5'):
    """Render an always-visible boxed callout (not collapsed)."""
    return HTML(
        f'<div style="margin: 1em 0; padding: 1em; border-left: 4px solid␣
 ↪{border_color}; '
        f'background: {bg_color}; line-height: 1.6;">{text}</div>'
    )

# --- Load all data ---
DATA = {}

with open(CYCLE / 'prereg' / 'rubric.json') as f:
    DATA['rubric'] = json.load(f)

with open(CYCLE / 'classify' / 'labeled_incidents_multimodel.json') as f:
    DATA['incidents'] = json.load(f)

prelabels = []
with open(CYCLE / 'calibration' / 'llm_prelabels.jsonl') as f:
    for line in f:
        prelabels.append(json.loads(line))
DATA['prelabels'] = prelabels

goldset = []
with open(CYCLE / 'calibration' / 'adjudicated_goldset.jsonl') as f:
    for line in f:
        goldset.append(json.loads(line))
DATA['goldset'] = goldset

precision_verif = []
with open(CYCLE / 'calibration' / 'precision_verification.jsonl') as f:
    for line in f:
        precision_verif.append(json.loads(line))
DATA['precision_verification'] = precision_verif

with open(CYCLE / 'calibration' / 'posteriors.json') as f:
    DATA['posteriors'] = json.load(f)
```

```
with open(CYCLE / 'calibration' / 'diagnostic.json') as f:
    DATA['diagnostic'] = json.load(f)

with open(CYCLE / 'infer' / 'inference_summary.json') as f:
    DATA['inference_summary'] = json.load(f)

DATA['lambda_samples'] = np.load(CYCLE / 'infer' / 'lambda_samples.npy')

with open(CYCLE / 'results' / 'concordance.json') as f:
    DATA['concordance'] = json.load(f)

with open(CYCLE / 'results' / 'selection_bias.json') as f:
    DATA['selection_bias'] = json.load(f)

with open(CYCLE / 'results' / 'rank_comparison_report.md') as f:
    DATA['rank_comparison_md'] = f.read()

# Build lookup tables
ENTRY_NAMES = {e['entry_id']: e['canonical_name'] for e in␣
 ↪DATA['rubric']['entries']}
INFER_ENTRY_ORDER = DATA['inference_summary']['entry_ids']

# Validate shapes
assert DATA['lambda_samples'].shape == (16000, 20), (
    f"Expected lambda_samples shape (16000, 20), got {DATA['lambda_samples'].
 ↪shape}"
)
assert len(INFER_ENTRY_ORDER) == 20, (
    f"Expected 20 entry_ids in inference_summary, got {len(INFER_ENTRY_ORDER)}"
)

print(f"Loaded {len(DATA)} data files. {len(DATA['incidents'])} incidents, "
      f"{len(DATA['prelabels'])} prelabels, {len(DATA['goldset'])}␣
 ↪adjudications, "
      f"{DATA['lambda_samples'].shape[0]} posterior draws. Ready.")
```

```
Loaded 12 data files. 6639 incidents, 5972 prelabels, 1200 adjudications, 16000
posterior draws. Ready.
```

1 What the Data Says About the 2026 Top 10

1.1 Act 1: The Question

The OWASP Top 10 for LLMs ranks AI security vulnerabilities. The 2025 list was built from expert surveys — hundreds of security professionals voting on what matters most. That process produced a consensus: Prompt Injection at #1, Sensitive Information Disclosure at #2, and so on down to #10.

Expert opinion is one signal. We wanted to check it against a second signal: the pattern of real-world incidents. We built a corpus of ~6,600 AI security incidents from public databases, classified each one against the 20-entry taxonomy, and asked: does the incident data agree with the experts?

This notebook walks through that analysis step by step. Along the way, you will see how the classification worked, how we measured its accuracy, and what a Bayesian model does with noisy measurements. Every chart and table below is computed live from the data — you can re-run any cell to verify.

Here are the 20 taxonomy entries we are working with. The "Incident Rank" column is blank for now. We will fill it in Act 6, after walking through the methodology.

[2]:
```
# Build the entry table with a placeholder rank column
entry_table = pd.DataFrame([
    {'#': i + 1, 'Entry ID': e['entry_id'], 'Name': e['canonical_name'],␣
 ↪'Incident Rank': '-'}
    for i, e in enumerate(DATA['rubric']['entries'])
])
entry_table = entry_table.set_index('#')
display(entry_table.style.set_caption(
    "20 taxonomy entries. Incident Rank will be filled in Act 6."
).set_properties(**{'text-align': 'left'}))
```

```
<pandas.io.formats.style.Styler at 0x336ed9d30>
```

1.2 Act 2: The Corpus

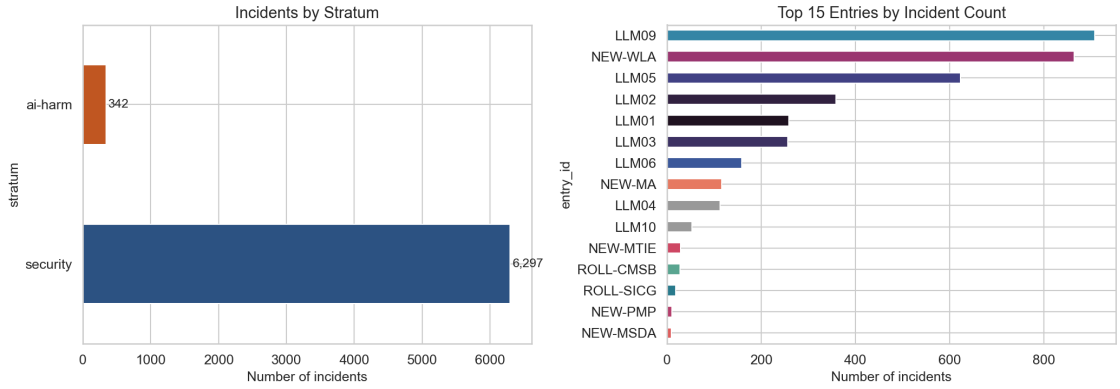
The corpus contains 6,639 incidents from public databases: CVE, GHSA, and OSV (security advisories), plus AIAAIC (a database of AI-related harms and controversies). Each record has a text description of what happened.

The corpus splits into two strata. The **security** stratum (CVE/GHSA/OSV) contains things like prompt injection exploits, data leakage through APIs, and supply chain compromises in ML packages. The **ai-harm** stratum (AIAAIC) contains things like algorithmic discrimination, deepfake misuse, and surveillance overreach.

This split matters. The classifier performs differently on each stratum — security incidents have more structured descriptions (CVE format), while ai-harm incidents are written as news summaries with varying detail.

[3]:
```
# Count incidents by stratum
inc_df = pd.DataFrame(DATA['incidents'])
stratum_counts = inc_df['stratum'].value_counts()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Bar chart by stratum
colors = ['#2c5282', '#c05621']
stratum_counts.plot.barh(ax=axes[0], color=colors[:len(stratum_counts)])
axes[0].set_title('Incidents by Stratum')
```

```
axes[0].set_xlabel('Number of incidents')
for i, (stratum, count) in enumerate(stratum_counts.items()):
    axes[0].text(count + 30, i, f'{count:,}', va='center', fontsize=11)

# Bar chart by entry (top 15)
entry_counts = inc_df[inc_df['entry_id'] != 'out-of-scope']['entry_id'].
 ↪value_counts().head(15)
bar_colors = [ENTRY_COLORS.get(eid, '#666666') for eid in entry_counts.index]
entry_counts.plot.barh(ax=axes[1], color=bar_colors)
axes[1].set_title('Top 15 Entries by Incident Count')
axes[1].set_xlabel('Number of incidents')
axes[1].invert_yaxis()

plt.tight_layout()
plt.show()
```



[4]:
```
# Show 2 real incident examples - one from each stratum
# Build stratum lookup from classified incidents (prelabels has no stratum␣
 ↪field)
_stratum_lookup = {inc['incident_id']: inc['stratum'] for inc in␣
 ↪DATA['incidents']}

prelabels_df = DATA['prelabels']
security_ex = next((p for p in prelabels_df if p['triage_tier'] == 'agree'
                    and p['consensus'] not in ('out-of-scope', None)
                    and _stratum_lookup.get(p['incident_id']) == 'security'),␣
 ↪None)
harm_ex = next((p for p in prelabels_df if p['triage_tier'] == 'agree'
                and _stratum_lookup.get(p['incident_id']) == 'ai-harm'
                and len(p['text']) > 100), None)
if security_ex is None or harm_ex is None:
```

```
    display(HTML('<p style="color: red;">Could not find suitable example␣
 ↪incidents.</p>'))

def incident_card(record, label):
    """Format an incident as an HTML card."""
    text = html.escape(shorten(record['text'], width=250, placeholder='...'))
    consensus = html.escape(str(record['consensus']))
    tier = html.escape(str(record['triage_tier']))
    inc_id = html.escape(str(record['incident_id']))
    return HTML(
        f'<div style="border: 1px solid #ccc; border-radius: 8px; padding: 1em;␣
 ↪'
        f'margin: 0.5em 0; background: #fafafa;">'
        f'<strong>{html.escape(label)}</strong><br>'
        f'<code>{inc_id}</code> · consensus: '
        f'<strong>{consensus}</strong> · tier: {tier}<br>'
        f'<p style="margin-top: 0.5em; color: #333;">{text}</p>'
        f'</div>'
    )

display(HTML('<h4>Example: Security stratum (CVE/GHSA/OSV)</h4>'))
display(incident_card(security_ex, 'Security incident'))

display(HTML('<h4>Example: AI-harm stratum (AIAAIC)</h4>'))
display(incident_card(harm_ex, 'AI-harm incident'))
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

[5]:
```
# F-frame sidebar
display(sidebar(
    'Deep dive: What the corpus cannot see (F-frame)',
    '<p>The corpus is built from a keyword crawl of public databases - CVE,␣
 ↪GHSA, OSV, '
    'and AIAAIC. Incidents that never became CVEs or harm-database entries are␣
 ↪invisible '
    'to us. A prompt injection attack against an internal enterprise tool that␣
 ↪was caught '
    'and patched quietly will never appear in this data.</p>'
    '<p>This creates structural bias toward vulnerability types that get␣
 ↪reported in '
    'public channels. Well-resourced organizations that fix issues internally␣
 ↪are '
```

```
    'underrepresented. Novel attack types that have not been assigned a CVE␣
 ↪category '
    'are invisible.</p>'
))

# F-circ callout - always visible, not collapsed
display(callout_box(
    '<strong>Structural limitation: taxonomy-frame circularity (F-circ)</
 ↪strong><br><br>'
    'We classified these incidents using the same taxonomy we are trying to␣
 ↪validate. '
    'If the classifier systematically favors certain entries, the incident␣
 ↪counts will '
    'appear to confirm the expert rankings even if the true pattern is␣
 ↪different. '
    'This is taxonomy-frame circularity. It means the concordance we measure␣
 ↪later '
    'is an upper bound on true agreement, not a precise estimate of it.',
    border_color='#d69e2e', bg_color='#fefcbf'
))
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

1.3 Act 3: Classification — How We Labeled 6,600 Incidents

Each incident was classified by three different large language models: Qwen 235B, Llama 405B, and DeepSeek V3. Each model independently read the incident text and assigned it to one of the 20 taxonomy entries — or marked it "out of scope" if none fit.

When all three models agreed on the same entry, we call it **agree tier**. When two agreed and one differed, **split tier**. When all three picked different entries, **disagree tier**.

The tier tells us how confident we can be in the classification. Agree-tier incidents have strong consensus. Disagree-tier incidents sit in ambiguous territory where even three independent classifiers could not converge.

[6]:
```
# Find a good agree-tier example with three matching votes
agree_ex = next((p for p in DATA['prelabels']
                 if p['triage_tier'] == 'agree'
                 and p['consensus'] not in ('out-of-scope', None)
                 and len(p['model_votes']) == 3
                 and len(p['text']) > 120), None)
assert agree_ex is not None, "No suitable agree-tier example found in prelabels"

display(HTML('<h4>Worked example: three-model classification</h4>'))
display(HTML(
    f'<div style="border: 1px solid #ccc; border-radius: 8px; padding: 1em; '
```

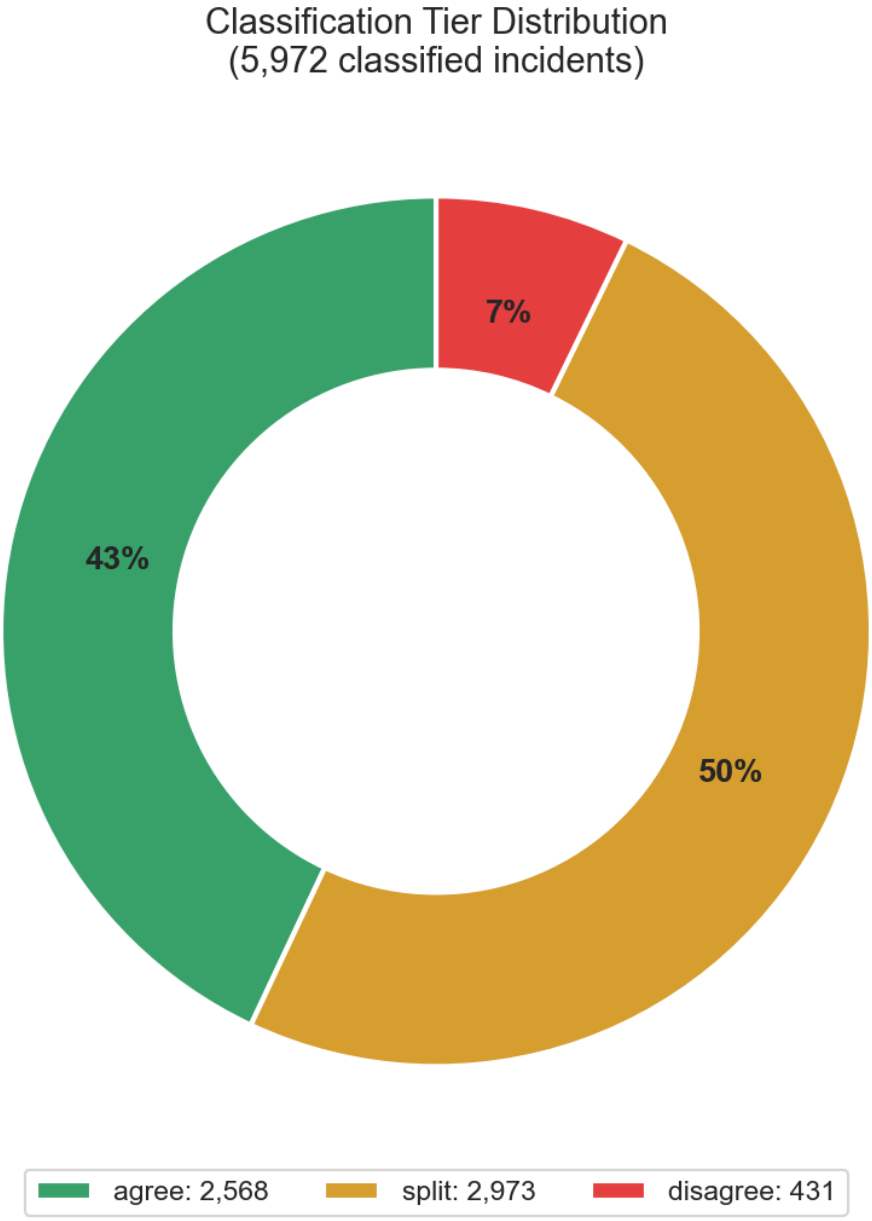
```
    f'margin: 0.5em 0; background: #f7fafc;">'
    f'<p style="color: #555;"><strong>Incident text</strong> (truncated):</p>'
    f'<p style="font-style: italic;">{html.escape(shorten(agree_ex["text"],␣
 ↪250, placeholder="..."))}</p>'
    f'<hr style="border: 0; border-top: 1px solid #e2e8f0;">'
    f'<table style="width: 100%; border-collapse: collapse;">'
    f'<tr style="background: #edf2f7;"><th>Model</th><th>Entry</
 ↪th><th>Confidence</th></tr>'
    + ''.join(
        f'<tr><td>{html.escape(v["model_id"].split("/")[-1])}</td>'
        f'<td><strong>{html.escape(v["entry_id"])}</strong></td>'
        f'<td>{v["confidence"]:.0%}</td></tr>'
        for v in agree_ex['model_votes']
    )
    + f'</table>'
    f'<p style="margin-top: 0.5em;">Consensus: <strong>{html.
 ↪escape(str(agree_ex["consensus"]))}</strong> '
    f'(tier: {html.escape(str(agree_ex["triage_tier"]))})</p>'
    f'</div>'
))
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

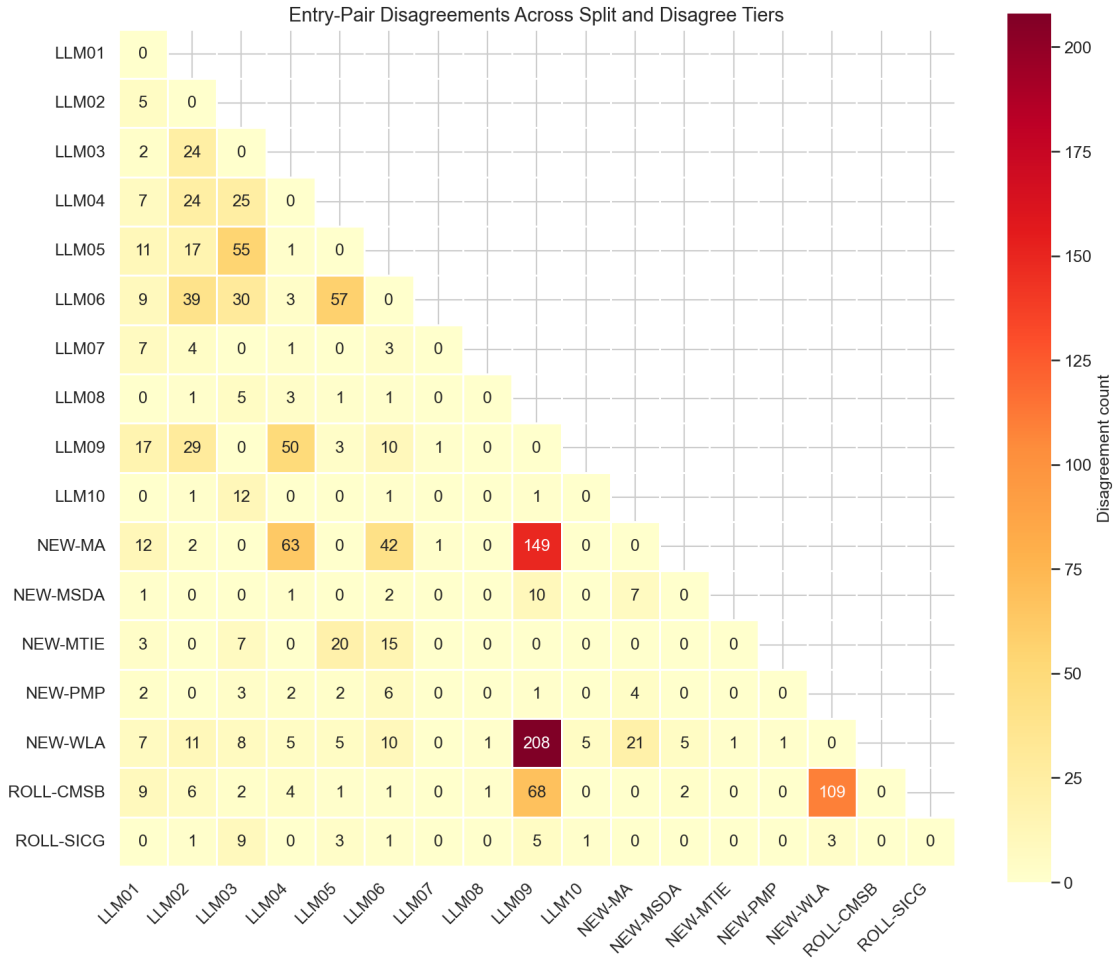
[7]:
```
# Tier distribution donut chart
tier_counts = Counter(p['triage_tier'] for p in DATA['prelabels'])
tier_labels = ['agree', 'split', 'disagree']
tier_values = [tier_counts.get(t, 0) for t in tier_labels]
tier_colors = ['#38a169', '#d69e2e', '#e53e3e']

fig, ax = plt.subplots(figsize=(7, 7))
wedges, texts, autotexts = ax.pie(
    tier_values, labels=None, autopct='%1.0f%%',
    colors=tier_colors, startangle=90, pctdistance=0.75,
    wedgeprops={'width': 0.4, 'edgecolor': 'white', 'linewidth': 2}
)
for t in autotexts:
    t.set_fontsize(13)
    t.set_fontweight('bold')

ax.legend(
    [f'{t}: {v:,}' for t, v in zip(tier_labels, tier_values)],
    loc='lower center', ncol=3, fontsize=11,
    bbox_to_anchor=(0.5, -0.05)
)
ax.set_title('Classification Tier Distribution\n(5,972 classified incidents)',␣
 ↪fontsize=14)
```

```
plt.tight_layout()
plt.show()
```



[8]:
```
# Build entry-pair disagreement matrix from split and disagree tiers
in_scope_entries = [eid for eid in ENTRY_IDS]
confusion = pd.DataFrame(0, index=in_scope_entries, columns=in_scope_entries)

for p in DATA['prelabels']:
    if p['triage_tier'] in ('split', 'disagree'):
```

```
        votes = [v['entry_id'] for v in p['model_votes'] if v['entry_id'] !=␣
 ↪'out-of-scope']
        unique_votes = list(set(votes))
        for i in range(len(unique_votes)):
            for j in range(i + 1, len(unique_votes)):
                a, b = unique_votes[i], unique_votes[j]
                if a in confusion.index and b in confusion.columns:
                    confusion.loc[a, b] += 1
                    confusion.loc[b, a] += 1

# Keep only entries with at least some disagreement
row_sums = confusion.sum(axis=1)
active = row_sums[row_sums > 5].index.tolist()
confusion_active = confusion.loc[active, active]

fig, ax = plt.subplots(figsize=(12, 10))
mask = np.triu(np.ones_like(confusion_active, dtype=bool), k=1)
sns.heatmap(
    confusion_active, mask=mask, cmap='YlOrRd', annot=True, fmt='d',
    linewidths=0.5, ax=ax, square=True, cbar_kws={'label': 'Disagreement count'}
)
ax.set_title('Entry-Pair Disagreements Across Split and Disagree Tiers',␣
 ↪fontsize=14)
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



[9]:
```
display(sidebar(
    'Deep dive: Why three models?',
    '<p>Single-model classification had lower precision in our early␣
 ↪experiments. '
    'A single model might confidently assign an incident to the wrong entry␣
 ↪because '
    'of biases in its training data or the phrasing of the prompt.</p>'
    '<p>Three models with majority vote reduces noise the same way a panel of␣
 ↪three '
    'judges reduces individual bias. If two of three models agree, we have␣
 ↪higher '
    'confidence in the label. The disagree tier (where all three pick different␣
 ↪entries) '
    'explicitly marks incidents where no classifier consensus exists.</p>'
))
```

```
display(sidebar(
    'Deep dive: The two-stage classification pipeline',
    '<p><strong>Stage 1 (heuristic):</strong> Regex and keyword indicators scan␣
 ↪the '
    'incident text for known patterns (e.g., "prompt injection," "CVE-2026-*").␣
 ↪'
    'This produces a fast initial assignment at low confidence (10%). All␣
 ↪incidents '
    'proceed to Stage 2 regardless of Stage 1 results.</p>'
    '<p><strong>Stage 2 (LLM):</strong> Each of three models reads the full␣
 ↪incident '
    'text alongside the complete rubric (all 20 entries with inclusion/
 ↪exclusion '
    'criteria). The model returns an entry_id, a confidence score, and a␣
 ↪rationale. '
    'The three Stage 2 labels are combined into the consensus and triage tier.</
 ↪p>'
))
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

1.4 Act 4: How Good Is the Classifier?

The classifier is a tool, not ground truth. To trust the incident counts, we need to measure how often the classifier gets it right — and how often it misses things.

Precision: When the classifier says "this incident belongs to LLM02," how often is it correct? We verified 323 classifications by hand to measure this. Each entry gets its own precision score — some entries are easier to classify than others.

Recall: Does the classifier find all incidents of a given type, or does it miss some? A human reviewer adjudicated 1,200 incidents across all tiers to measure this. The reviewer saw the incident text and the three model votes, then decided whether to accept the consensus, override it, or mark the incident as out of scope.

1.4.1 Why precision varies so much across entries

The chart below shows precision ranging from 93% (LLM01, LLM03) down to 13% (LLM08). The variation is not random — it reflects how cleanly each entry's definition separates it from neighboring categories. Four entries fall **below the 50% threshold**, which deserves explanation:

- **LLM08 (Vector and Embedding Weaknesses): 13%.** This is the lowest precision in the taxonomy. Out of every 8 incidents the classifier labels as LLM08, only 1 actually is. The category describes a narrow class of attacks — adversarial manipulation of embedding spaces, vector database poisoning, retrieval-augmented generation exploits. But the classifier confuses it with LLM03 (Training Data Poisoning) and general data integrity issues. Most incidents classified here describe data manipulation that affects a model, which is conceptually adjacent but taxonomically distinct.

- **LLM07 (System Prompt Leakage): 31%.** The classifier struggles to distinguish "extracting a system prompt" (LLM07) from "overriding a system prompt" (LLM01, Prompt Injection). Both involve adversarial interaction with the prompt layer, and many real incidents involve both — an attacker extracts the system prompt *in order to* craft a better injection. The boundary is taxonomically clear but operationally blurred.
- **ROLL-CFAS (Comprehensive Framework Attacks): 33%.** Only 3 precision observations — the posterior is dominated by the Beta(1,1) prior, so the 33% estimate has a 90% CI stretching from 3% to 78%. The category's broad definition ("attacks on the comprehensive AI framework") makes it a natural catch-all that absorbs incidents from adjacent entries.
- **ROLL-CMSB (Cross-Modal Safety Bypass): 44%.** This entry sits at the center of the confusion boundary described in Act 9B. A deepfake video that bypasses content filters could be classified as ROLL-CMSB (cross-modal bypass), LLM09 (misinformation), or NEW-WLA (weaponized abuse). The classifier picks one; a human might reasonably pick any of the three.

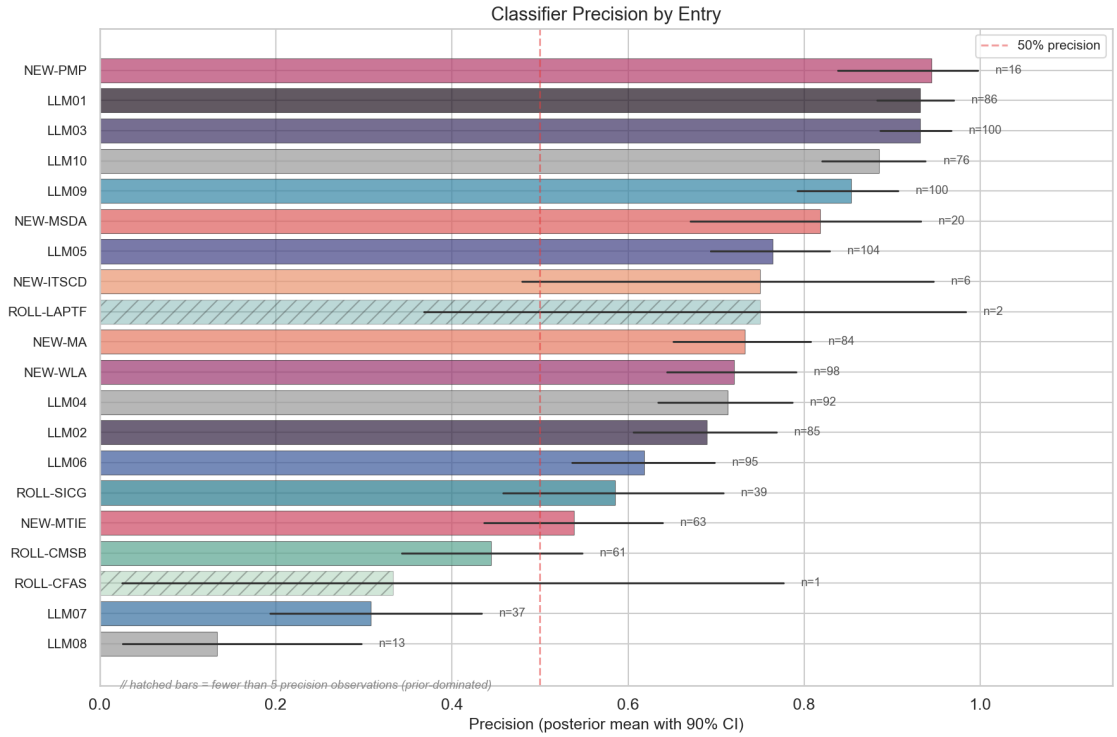
1.4.2 What the 50% threshold means

Precision below 50% means the classifier is **wrong more often than it is right** for that entry. When you see an incident labeled "LLM08," the odds are 7:1 against it actually being a vector/embedding weakness. This has direct consequences for the Bayesian model in Act 5: low-precision entries get large upward corrections (because much of their observed count is misclassification noise from other entries) and wide credible intervals (because the correction itself is uncertain). A 13% precision estimate does not mean the entry is unimportant — it means the *automated measurement* of that entry is unreliable, and the model's uncertainty reflects that.

Stratum coverage note: The 323 precision verifications were drawn entirely from the security stratum (CVE/GHSA/OSV). The ai-harm stratum (AIAAIC) has no precision measurements — the Bayesian model uses a flat Beta(1,1) prior for ai-harm precision, meaning it assumes no prior knowledge about classifier accuracy on those incidents (prior mean 0.5). This means error correction for ai-harm incidents relies on borrowed estimates, not direct measurement.

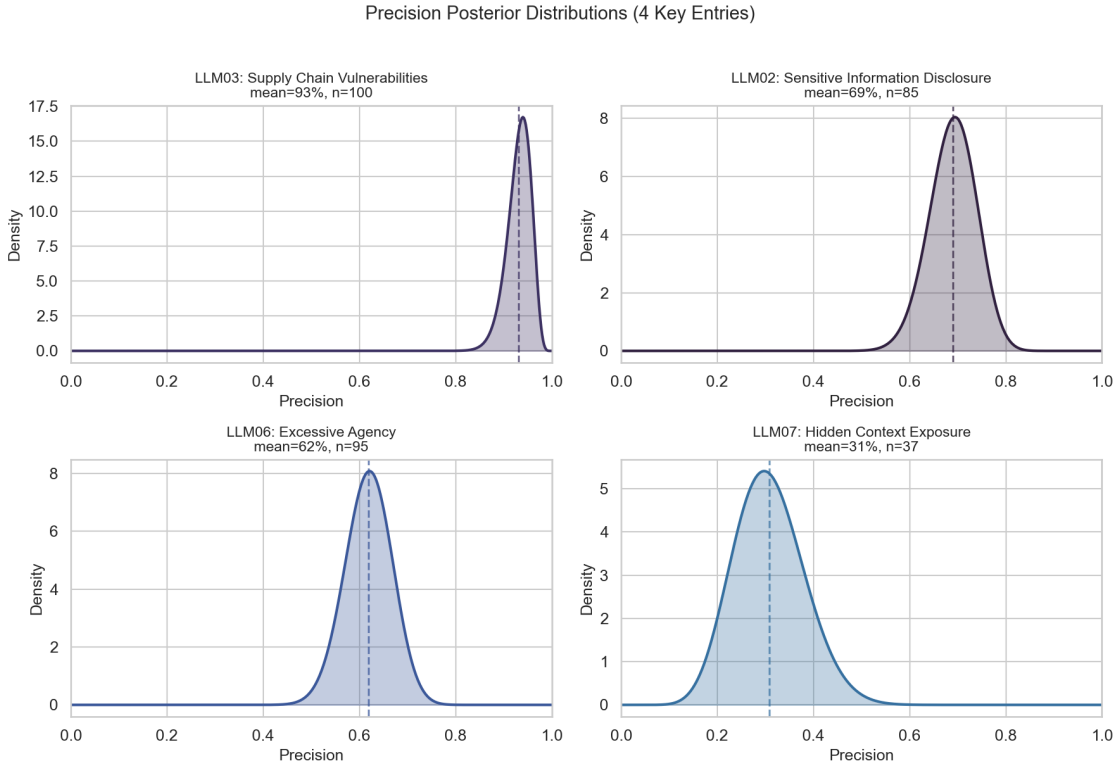
[10]:
```
# Compute precision posterior means from posteriors.json
precision_data = DATA['posteriors']['precision']
diag = DATA['diagnostic']['entry_reports']

prec_rows = []
for key, params in precision_data.items():
    entry_id = key.split('::')[0]
    if entry_id == 'out-of-scope':
        continue
    alpha, beta = params['alpha'], params['beta']
    mean = alpha / (alpha + beta)
    n = int(alpha + beta - 2)  # subtract prior pseudocounts (1+1)
    prec_rows.append({
        'entry_id': entry_id,
        'precision_mean': mean,
        'sample_size': n,
        'alpha': alpha,
```

```
        'beta': beta,
    })

prec_df = pd.DataFrame(prec_rows).sort_values('precision_mean', ascending=True)

fig, ax = plt.subplots(figsize=(12, 8))

# Compute 90% credible intervals
for i, row in enumerate(prec_df.itertuples()):
    ci_low, ci_high = stats.beta.ppf([0.05, 0.95], row.alpha, row.beta)
    color = ENTRY_COLORS.get(row.entry_id, '#666666')
    hatch = '//' if row.sample_size < 5 else ''

    ax.barh(i, row.precision_mean, color=color, alpha=0.7 if row.sample_size >=␣
 ↪5 else 0.35,
            edgecolor='#333', linewidth=0.5, hatch=hatch)
    ax.plot([ci_low, ci_high], [i, i], color='#333', linewidth=1.5,␣
 ↪solid_capstyle='round')
    ax.text(ci_high + 0.02, i, f'n={row.sample_size}', va='center', fontsize=9,␣
 ↪color='#555')

ax.set_yticks(range(len(prec_df)))
ax.set_yticklabels([f'{row.entry_id}' for row in prec_df.itertuples()],␣
 ↪fontsize=10)
ax.set_xlabel('Precision (posterior mean with 90% CI)')
ax.set_title('Classifier Precision by Entry', fontsize=14)
ax.set_xlim(0, 1.15)
ax.axvline(0.5, color='#e53e3e', linestyle='--', alpha=0.5, label='50%␣
 ↪precision')

# Add hatched-bar footnote
ax.text(0.02, -1.5, '// hatched bars = fewer than 5 precision observations␣
 ↪(prior-dominated)',
        fontsize=9, color='#888', style='italic', transform=ax.
 ↪get_yaxis_transform())

ax.legend(fontsize=10)
plt.tight_layout()
plt.show()
```



[11]:
```
# Show Beta posterior distributions for 4 key entries
key_entries = ['LLM03', 'LLM02', 'LLM06', 'LLM07']
x = np.linspace(0, 1, 500)

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for ax, eid in zip(axes.flat, key_entries):
    key = f'{eid}::security'
    if key not in precision_data:
        continue
    alpha = precision_data[key]['alpha']
    beta_param = precision_data[key]['beta']
    y = stats.beta.pdf(x, alpha, beta_param)
    mean = alpha / (alpha + beta_param)
    n = int(alpha + beta_param - 2)

    color = ENTRY_COLORS.get(eid, '#666666')
    ax.fill_between(x, y, alpha=0.3, color=color)
    ax.plot(x, y, color=color, linewidth=2)
    ax.axvline(mean, color=color, linestyle='--', alpha=0.7)
    ax.set_title(f'{eid}: {ENTRY_NAMES[eid]}\nmean={mean:.0%}, n={n}',␣
 ↪fontsize=11)
    ax.set_xlabel('Precision')
    ax.set_ylabel('Density')
```

```
    ax.set_xlim(0, 1)

plt.suptitle('Precision Posterior Distributions (4 Key Entries)', fontsize=14,␣
 ↪y=1.02)
plt.tight_layout()
plt.show()
```



[12]:
```
display(sidebar(
    'Deep dive: The gold-set process - 1,200 human adjudications',
    '<p>A human reviewer (the project author) adjudicated 1,200 incidents using␣
 ↪a '
    'blind-first protocol. For each incident:</p>'
    '<ol>'
    '<li>Read the incident text without seeing the model votes (blind label).</
 ↪li>'
    '<li>Record an independent classification.</li>'
    '<li>Then reveal the three model votes and the consensus.</li>'
    '<li>Make a final decision: accept the consensus, override to a different␣
 ↪entry, '
    'assign multiple labels, or mark as out of scope.</li>'
    '</ol>'
```

```
    '<p>"Adjudication" is different from voting. The reviewer is not adding a␣
 ↪fourth '
    'opinion - they are making a judgment call after seeing both the text and␣
 ↪the '
    'model reasoning. The blind label (step 2) guards against anchoring to the '
    'model consensus.</p>'
))

# Full precision posteriors table
prec_table_rows = []
for key, params in sorted(precision_data.items()):
    entry_id = key.split('::')[0]
    if entry_id == 'out-of-scope':
        continue
    alpha, beta_p = params['alpha'], params['beta']
    mean = alpha / (alpha + beta_p)
    ci_low, ci_high = stats.beta.ppf([0.05, 0.95], alpha, beta_p)
    n = int(alpha + beta_p - 2)
    flag = '(prior-dominated)' if n < 5 else ''
    prec_table_rows.append({
        'Entry': entry_id,
        'alpha': alpha, 'beta': beta_p,
        'Mean': f'{mean:.1%}',
        '90% CI': f'[{ci_low:.1%}, {ci_high:.1%}]',
        'n': n,
        'Note': flag,
    })

prec_table_html = pd.DataFrame(prec_table_rows).to_html(index=False,␣
 ↪escape=False)
display(sidebar('Deep dive: Full precision posteriors table', prec_table_html))
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

1.5 Act 5: From Counts to Rankings — The Bayesian Model

Raw incident counts would be misleading. An entry whose classifier has 30% precision looks like it has many incidents — but two-thirds of those are misclassifications wrongly attributed to it.

We need a model that adjusts the observed counts for known classifier error. Think of a bathroom scale that reads 2 pounds heavy. You would subtract 2 pounds from every reading. The Bayesian model does this for each entry separately, and it carries the uncertainty through — if the scale is 2 ± 1 pounds off, the corrected weight is also uncertain.

1.5.1 What happens with low-precision entries

For entries above 50% precision, the correction is a moderate downward adjustment — some of the observed incidents were misclassified, so the true count is lower than the raw count. The correction tightens toward the true signal.

For entries **below 50% precision**, the correction works differently. If only 13% of incidents labeled "LLM08" actually belong there, the model must infer the true rate from a signal that is mostly noise. It's like reading a bathroom scale that is off by more than half the measurement — the "correction" is larger than the reading itself. This produces two effects: (1) the corrected estimate can be very different from the raw count, and (2) the uncertainty around that estimate is wide, because small changes in the precision estimate propagate into large changes in the corrected rate.

This is why some entries in the Act 6 chart have 90% credible intervals spanning 10+ rank positions. The width is not a flaw in the model — it is the model honestly reporting how much information the data contains about each entry's true incident rate.

1.5.2 The model's inputs

The model takes the observed incident counts, the measured precision and recall for each entry, and produces a **posterior distribution** over the true incident rate for each entry. A posterior distribution is not a single number — it is a range of plausible values given the data. Wide distributions mean less certainty.

We drew 16,000 samples from this distribution using Markov chain Monte Carlo (MCMC). MCMC is a method for sampling from probability distributions that are too complex to compute directly. It generates a sequence of random samples that, after enough iterations, represents the target distribution. Our run used 4 chains of 4,000 samples each, with 2,000 warmup iterations per chain.

1.5.3 Handling missing data

Three entries — LLM04, LLM08, LLM10 — are **frame-blind**: their incident counts come entirely from one stratum, so the model cannot cross-validate their rates across strata. These entries are included in the posterior but flagged with a in the charts. Their rank estimates carry additional structural uncertainty beyond what the credible intervals capture.

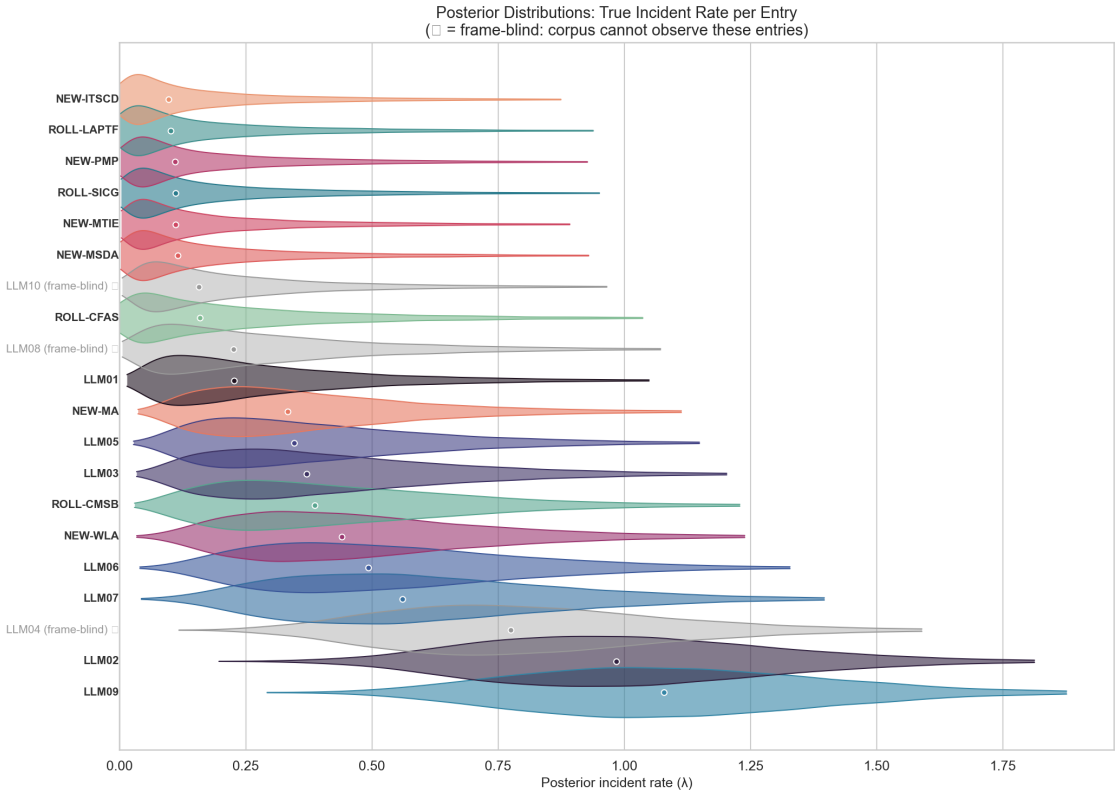
For 16 of 20 entries in the ai-harm stratum, we have not measured recall directly. The model uses a conservative prior estimate of ~1% recall for those entries — Beta(1, 101). This means the model assumes the classifier finds very few of those incidents and adjusts upward accordingly. These corrections are large, which is one reason the credible intervals in Act 6 are wide.

Precision in the ai-harm stratum is also unmeasured. The model uses a flat Beta(1,1) = Uniform(0,1) prior, meaning it treats ai-harm precision as completely unknown (prior mean 50%). This is a weaker assumption than the security stratum, where we have 5–88 hand-verified observations per entry.

[13]:
```
# Ridge / violin plot of posterior lambda for all 20 entries
lambda_samples = DATA['lambda_samples']
entry_order = INFER_ENTRY_ORDER

# Compute medians for sorting
```

```
medians = np.median(lambda_samples, axis=0)
sort_idx = np.argsort(medians)[::-1]

fig, ax = plt.subplots(figsize=(14, 10))

for plot_i, data_i in enumerate(sort_idx):
    eid = entry_order[data_i]
    samples = lambda_samples[:, data_i]
    is_frame_blind = eid in FRAME_BLIND

    # KDE for the ridge
    kde_x = np.linspace(samples.min(), np.percentile(samples, 99), 300)
    try:
        kde = stats.gaussian_kde(samples)
        kde_y = kde(kde_x)
    except Exception:
        continue

    # Normalize height
    kde_y = kde_y / kde_y.max() * 0.8

    color = '#999999' if is_frame_blind else ENTRY_COLORS.get(eid, '#4a86c8')
    alpha = 0.4 if is_frame_blind else 0.6

    ax.fill_between(kde_x, plot_i - kde_y, plot_i + kde_y,
                    alpha=alpha, color=color, linewidth=0)
    ax.plot(kde_x, plot_i + kde_y, color=color, linewidth=1)
    ax.plot(kde_x, plot_i - kde_y, color=color, linewidth=1)

    # Mark median
    med = medians[data_i]
    ax.plot(med, plot_i, 'o', color='white', markersize=5, zorder=5)
    ax.plot(med, plot_i, 'o', color=color, markersize=3, zorder=6)

    # Label
    label = f'{eid}'
    if is_frame_blind:
        label += ' (frame-blind)  '
    ax.text(-0.001, plot_i, label, ha='right', va='center', fontsize=10,
            color='#999' if is_frame_blind else '#333',
            fontweight='normal' if is_frame_blind else 'bold')

ax.set_yticks([])
ax.set_xlabel('Posterior incident rate ( )', fontsize=12)
ax.set_title('Posterior Distributions: True Incident Rate per Entry\n'
             '(  = frame-blind: corpus cannot observe these entries)',␣
 ↪fontsize=14)
```

```
ax.set_xlim(left=-0.0005)

plt.subplots_adjust(left=0.22)
plt.tight_layout()
plt.show()
```



[14]:
```
# Summary statistics from lambda_samples and diagnostic.json
summary_rows = []
for i, eid in enumerate(INFER_ENTRY_ORDER):
    samples = lambda_samples[:, i]
    med = np.median(samples)
    ci_low, ci_high = np.percentile(samples, [5, 95])
    diag_report = DATA['diagnostic']['entry_reports'].get(eid, {})
    flag = diag_report.get('flag', '-')

    summary_rows.append({
        'Entry': eid,
        'Name': ENTRY_NAMES.get(eid, ''),
        '_median_numeric': med,
        'Median  ': f'{med:.4f}',
        '90% CI': f'[{ci_low:.4f}, {ci_high:.4f}]',
```

```
        'CI Width': f'{ci_high - ci_low:.4f}',
        'Diagnostic': flag,
    })

summary_df = pd.DataFrame(summary_rows).sort_values('_median_numeric',␣
↪ascending=False)
summary_df = summary_df.drop(columns=['_median_numeric'])
display(summary_df.style
    .set_caption('Posterior summary: median incident rate, 90% credible␣
↪interval, diagnostic flag')
    .set_properties(**{'text-align': 'left'})
    .apply(lambda row: ['background: #f0f0f0' if row['Entry'] in FRAME_BLIND␣
↪else ''
                        for _ in row], axis=1)
)
```

<pandas.io.formats.style.Styler at 0x34b06e0d0>

[15]:
```
# NumPyro model specification sidebar
model_source = (REPO_ROOT / 'engine' / 'model' / 'inference.py').read_text()
# Extract just the model function
model_start = model_source.find('def model(')
model_end = model_source.find('\n    # ------', model_start + 1)
if model_end == -1:
    model_end = model_source.find('\n    try:', model_start)
model_code = model_source[model_start:model_end]

display(sidebar(
    'Deep dive: The NumPyro model specification',
    '<p>This is the actual model we used, written in NumPyro (a probabilistic '
    'programming library for JAX). You do not need to install NumPyro to run␣
↪this '
    'notebook - the results above are pre-computed.</p>'
    f'<pre style="background: #1a202c; color: #e2e8f0; padding: 1em; '
    f'border-radius: 4px; overflow-x: auto; font-size: 0.85em;">'
    f'{model_code.replace("<", "&lt;").replace(">", "&gt;")}</pre>'
    '<p><strong>Key components:</strong></p>'
    '<ul>'
    '<li><code>lambda</code>: latent prevalence per entry (HalfNormal prior)</
↪li>'
    '<li><code>recall</code>, <code>precision</code>: per-entry, per-stratum '
    'measurement error (Beta priors from gold-set calibration)</li>'
    '<li><code>concentration</code>: over-dispersion parameter (Gamma prior)</
↪li>'
    '<li>The FP leakage term (<code>einsum</code>) accounts for misclassified '
    'incidents that spill from one entry into another</li>'
    '<li>Negative-Binomial likelihood handles count over-dispersion</li>'
```

```
    '</ul>'
))

# MCMC diagnostics sidebar
inf_summary = DATA['inference_summary']
max_rhat = max(v for k, v in inf_summary['r_hat'].items() if k.
 ↪startswith('lambda'))
min_ess = min(v for k, v in inf_summary['ess'].items() if k.
 ↪startswith('lambda'))
total_draws = inf_summary['num_samples'] * inf_summary['num_chains']

display(sidebar(
    'Deep dive: MCMC convergence diagnostics',
    f'<p>The MCMC sampler ran <strong>{inf_summary["num_chains"]} chains</
 ↪strong>, '
    f'each with <strong>{inf_summary["num_warmup"]:,}</strong> warmup␣
 ↪iterations '
    f'and <strong>{inf_summary["num_samples"]:,}</strong> sampling iterations, '
    f'producing <strong>{total_draws:,}</strong> posterior draws total.</p>'
    '<p><strong>R-hat</strong> measures whether the chains converged to the␣
 ↪same '
    'distribution. Values near 1.0 mean convergence. Our maximum R-hat across␣
 ↪all '
    f'lambda parameters: <strong>{max_rhat:.6f}</strong> (threshold: 1.01). '
    'All parameters pass.</p>'
    f'<p><strong>Effective sample size (ESS)</strong> measures how many␣
 ↪independent '
    f'samples the chains produced. Our minimum ESS for lambda parameters: '
    f'<strong>{min_ess:,.0f}</strong> out of {total_draws:,} draws '
    f'({min_ess/total_draws:.0%} efficiency). Higher is better.</p>'
    f'<p><strong>Divergences</strong>: <strong>{inf_summary["divergences"]}</
 ↪strong>. '
    'Zero divergences means the sampler explored the posterior geometry without␣
 ↪'
    'numerical problems. Any non-zero count would indicate regions the sampler '
    'could not traverse reliably.</p>'
))
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

1.6 Act 6: The Incident-Derived Rankings

These rankings reflect what the incident data suggests after correcting for classifier error. They are one signal, not the final word.

For each entry, the Bayesian model gives us a posterior distribution over its true incident rate. We

rank entries by their median rate and report a 90% credible interval on the rank. Some entries have tight intervals (the data is informative) and others are wide (less certain). The width tells you how much to trust the rank position.

1.6.1 How to read the chart

Each row is one taxonomy entry. The diamond marks the **median rank** — the rank position at the center of the posterior distribution. The horizontal bar spans the **90% credible interval** — the range of ranks that the model considers plausible given the data and its uncertainty about precision, recall, and the true incident rate.

Tight intervals (e.g., LLM02 spanning 1–6) mean the data strongly constrains that entry's position. Even after accounting for classifier error, the evidence points to a narrow range.

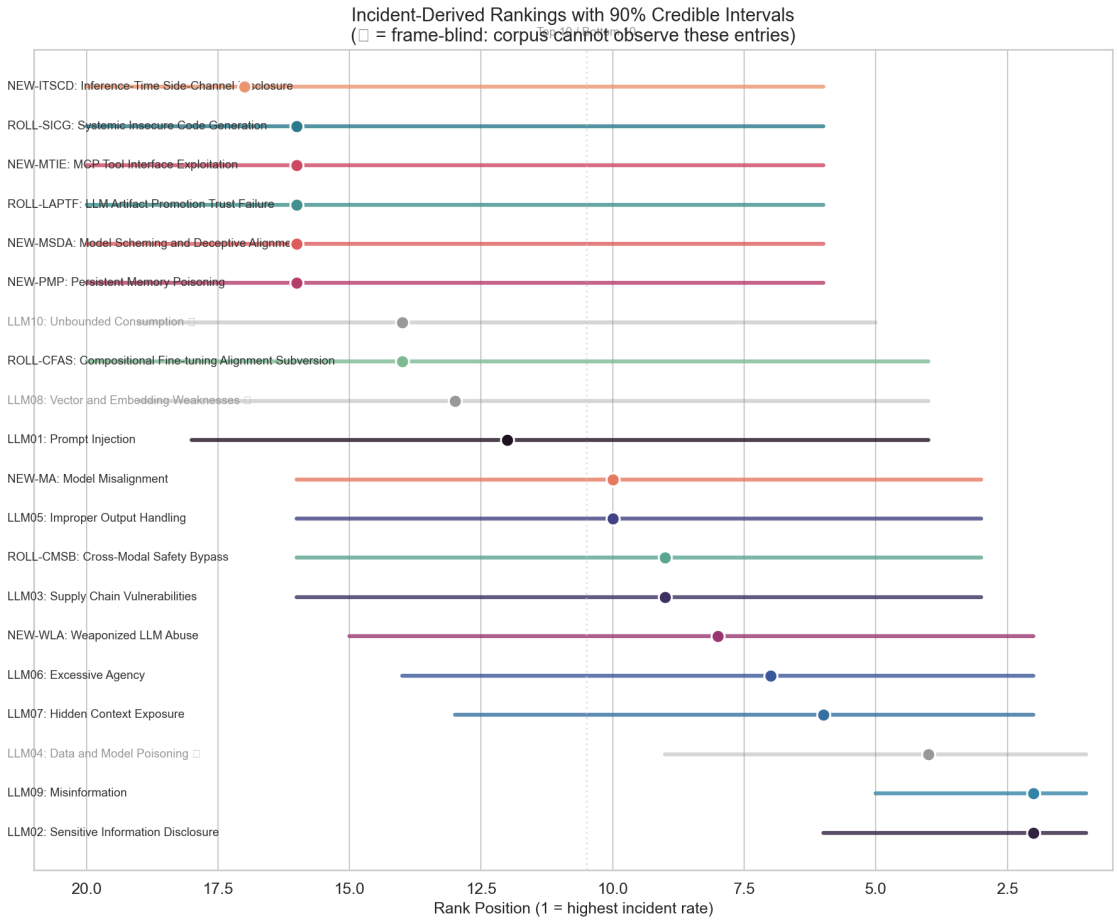
Wide intervals (e.g., spanning 6–20) mean the data is compatible with many rank positions. This happens when the entry has low precision (the correction is large and uncertain), few observations (small sample sizes produce wide posteriors), or unmeasured recall (the model uses a conservative prior that adds uncertainty).

Grey entries (LLM04, LLM08, LLM10) are frame-blind — their measurements come from only one stratum. Their positions carry structural uncertainty beyond what the CI captures.

The static matplotlib chart shows the rank distributions. The interactive plotly chart below it lets you hover over each entry for details: median rank, CI bounds, diagnostic flag, and frame-blind status.

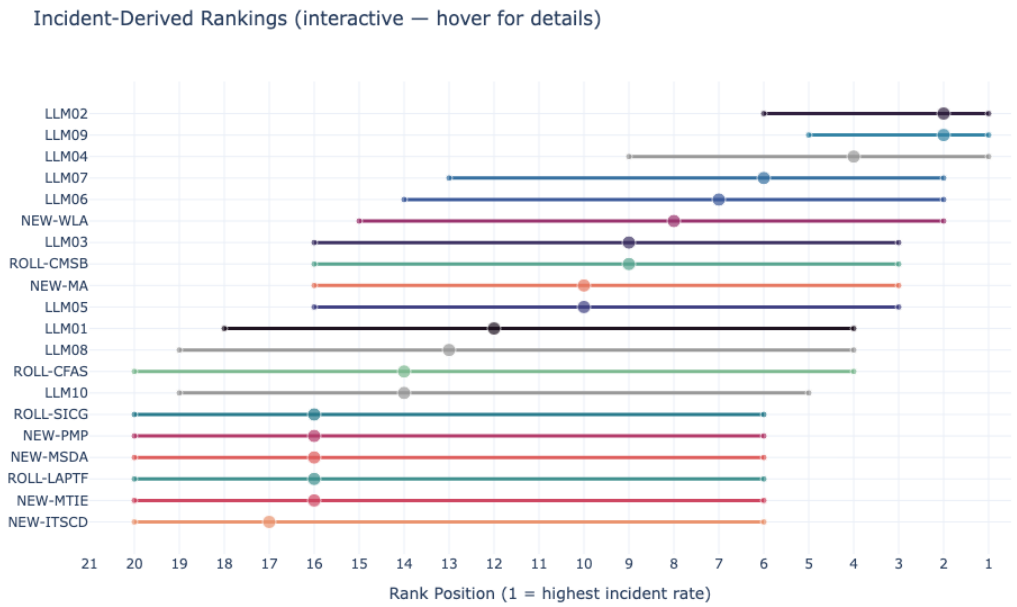
[16]:
```
# Compute rank distributions from lambda samples
lambda_samples = DATA['lambda_samples']
entry_order = INFER_ENTRY_ORDER

# For each posterior draw, rank entries (1=highest rate)
# argsort gives indices that sort ascending; we want descending
ranks = np.zeros_like(lambda_samples, dtype=int)
for draw in range(lambda_samples.shape[0]):
    order = np.argsort(-lambda_samples[draw])
    for rank, idx in enumerate(order):
        ranks[draw, idx] = rank + 1

# Compute median rank and 90% CI for each entry
rank_stats = []
for i, eid in enumerate(entry_order):
    r = ranks[:, i]
    rank_stats.append({
        'entry_id': eid,
        'name': ENTRY_NAMES.get(eid, eid),
        'median_rank': np.median(r),
        'ci_low': np.percentile(r, 5),
        'ci_high': np.percentile(r, 95),
        'frame_blind': eid in FRAME_BLIND,
    })
```

```
rank_df = pd.DataFrame(rank_stats).sort_values('median_rank')

# Dumbbell chart
fig, ax = plt.subplots(figsize=(12, 10))

for i, row in enumerate(rank_df.itertuples()):
    color = '#999999' if row.frame_blind else ENTRY_COLORS.get(row.entry_id,␣
 ↪'#4a86c8')
    alpha_val = 0.4 if row.frame_blind else 0.8

    # CI bar
    ax.plot([row.ci_low, row.ci_high], [i, i],
            color=color, linewidth=3, alpha=alpha_val, solid_capstyle='round')
    # Median dot
    ax.plot(row.median_rank, i, 'o', color=color, markersize=10,
            markeredgecolor='white', markeredgewidth=1.5, zorder=5)

    # Label
    label = f'{row.entry_id}: {row.name}'
    if row.frame_blind:
        label += '  '
    ax.text(21.5, i, label, va='center', fontsize=9,
            color='#999' if row.frame_blind else '#333')

ax.set_yticks([])
ax.set_xlabel('Rank Position (1 = highest incident rate)', fontsize=12)
ax.set_xlim(0.5, 21)
ax.invert_xaxis()
ax.set_title('Incident-Derived Rankings with 90% Credible Intervals\n'
             '(  = frame-blind: corpus cannot observe these entries)',␣
 ↪fontsize=14)
ax.axvline(10.5, color='#ccc', linestyle=':', alpha=0.5)
ax.text(10.5, len(rank_df) + 0.3, 'Top 10 / Bottom 10', ha='center',
        fontsize=9, color='#999')

plt.subplots_adjust(left=0.22)
plt.tight_layout()
plt.show()
```



[17]:
```
from IPython.display import Image as _StaticImage
# Interactive plotly version with hover detail
rank_df_sorted = rank_df.sort_values('median_rank')

fig = go.Figure()

for _, row in rank_df_sorted.iterrows():
    eid = row['entry_id']
    color = '#999999' if row['frame_blind'] else f'rgb{tuple(int(c*255) for c␣
 ↪in ENTRY_COLORS.get(eid, (0.3, 0.5, 0.8))[:3])}' if isinstance(ENTRY_COLORS.
 ↪get(eid), tuple) else '#4a86c8'

    # Get diagnostic flag
    diag_flag = DATA['diagnostic']['entry_reports'].get(eid, {}).get('flag', '-
')

    fig.add_trace(go.Scatter(
        x=[row['ci_low'], row['median_rank'], row['ci_high']],
```

```
        y=[eid, eid, eid],
        mode='lines+markers',
        marker=dict(size=[6, 12, 6], color=color),
        line=dict(color=color, width=3),
        name=eid,
        hovertemplate=(
            f'<b>{eid}: {row["name"]}</b><br>'
            f'Median rank: {row["median_rank"]:.0f}<br>'
            f'90% CI: [{row["ci_low"]:.0f}, {row["ci_high"]:.0f}]<br>'
            f'Diagnostic: {diag_flag}<br>'
            f'Frame-blind: {"Yes  " if row["frame_blind"] else "No"}'
            '<extra></extra>'
        ),
        showlegend=False,
    ))

fig.update_layout(
    title='Incident-Derived Rankings (interactive - hover for details)',
    xaxis_title='Rank Position (1 = highest incident rate)',
    xaxis=dict(range=[21, 0.5], dtick=1),
    yaxis=dict(autorange='reversed'),
    height=600,
    margin=dict(l=100),
)
# Render as static PNG only - preserves PDF compatibility and avoids the
# nbconvert 'html-only mimetype' warning on plotly interactive HTML.
display(_StaticImage(fig.to_image(format='png', width=1000, height=600)))
```



[18]:
```
# Fill in the "?" column from Act 1
updated_table = pd.DataFrame([
    {
        '#': i + 1,
        'Entry ID': e['entry_id'],
        'Name': e['canonical_name'],
        'Incident Rank':␣
 ↪f"{rank_df[rank_df['entry_id']==e['entry_id']]['median_rank'].values[0]:.0f}"
                        if len(rank_df[rank_df['entry_id']==e['entry_id']]) >␣
 ↪0 else '-',
    }
    for i, e in enumerate(DATA['rubric']['entries'])
])
updated_table = updated_table.set_index('#')
display(updated_table.style.set_caption(
    "The table from Act 1, now with incident-derived ranks filled in."
).set_properties(**{'text-align': 'left'}))
```

```
<pandas.io.formats.style.Styler at 0x34c9b0410>
```

1.7 Act 7: The Confrontation — Do Experts and Incidents Agree?

Cohen's weighted kappa measures agreement between two ranking systems, adjusted for chance. A value of 1.0 means perfect agreement. A value of 0 means no better than random. Negative values mean systematic disagreement.

Our result: kappa = 0.20, with a 90% credible interval of [−0.16, 0.57]. This interval includes zero. We cannot exclude the possibility that expert and incident rankings agree by chance alone. The point estimate of 0.20 suggests slight agreement, but the wide interval means this is a weak signal, not a firm conclusion.

Why is the interval so wide? Two reasons. First, we only have 17 measurable entries (three are frame-blind). Statistical agreement measures need larger samples for narrow confidence intervals. Second, the posterior rank distributions themselves are wide — most entries have 90% CIs spanning 10+ rank positions.

Five entries have posterior probability of tier mismatch exceeding 83% — meaning that across the full joint posterior, the Bayesian model and expert survey place these entries in different thirds of the ranking more than 83% of the time:

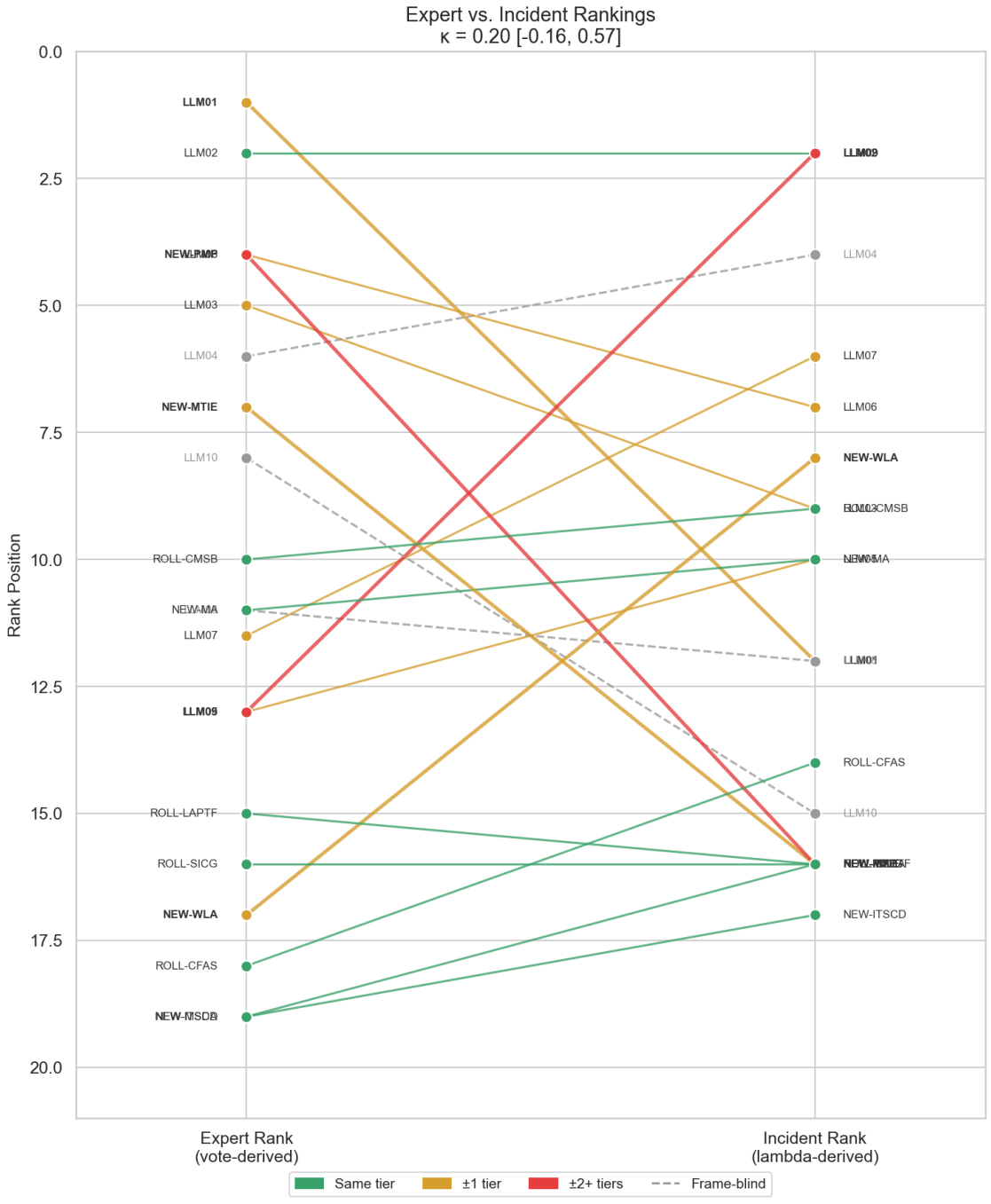
- **LLM01 Prompt Injection**: experts rank it #1 (90% CI: 1–2), incidents rank it #12 (90% CI: 4–18)
- **LLM09 Misinformation**: incidents rank it #2 (90% CI: 1–5), experts rank it #13 (90% CI: 9–16)
- **NEW-MTIE MCP Tool Interface Exploitation**: experts rank it #7 (90% CI: 5–9), incidents #16 (90% CI: 6–20)

- **NEW-PMP Persistent Memory Poisoning**: experts rank it #4 (90% CI: 2–7), incidents #16 (90% CI: 6–20)
- **NEW-WLA Weaponized LLM Abuse**: incidents rank it #8 (90% CI: 3–15), experts #17 (90% CI: 13–20)

[19]:
```
# Parse rank comparison report for vote ranks
lines = DATA['rank_comparison_md'].strip().split('\n')
comparison_rows = []
for line in lines:
    if line.startswith('|') and not line.startswith('|---') and 'Entry' not in␣
 ↪line:
        parts = [p.strip() for p in line.split('|')[1:-1]]
        if len(parts) >= 6:
            eid = parts[0]
            try:
                lambda_rank = float(parts[1].split('(')[0].strip())
                vote_rank = float(parts[2].split('(')[0].strip())
                tier_agreement = parts[3]
                direction = parts[4]
            except (ValueError, IndexError):
                continue
            comparison_rows.append({
                'entry_id': eid,
                'lambda_rank': lambda_rank,
                'vote_rank': vote_rank,
                'tier_agreement': tier_agreement,
                'direction': direction,
            })

comp_df = pd.DataFrame(comparison_rows)

# Concordance data
conc = DATA['concordance']
flags = {f['entry_id'] for f in conc['flags']}

fig, ax = plt.subplots(figsize=(10, 12))

for _, row in comp_df.iterrows():
    eid = row['entry_id']
    is_fb = eid in FRAME_BLIND
    is_flagged = eid in flags

    # Color by tier agreement
    if row['tier_agreement'] == 'same':
        color = '#38a169'
    elif row['tier_agreement'] == '±1':
        color = '#d69e2e'
```

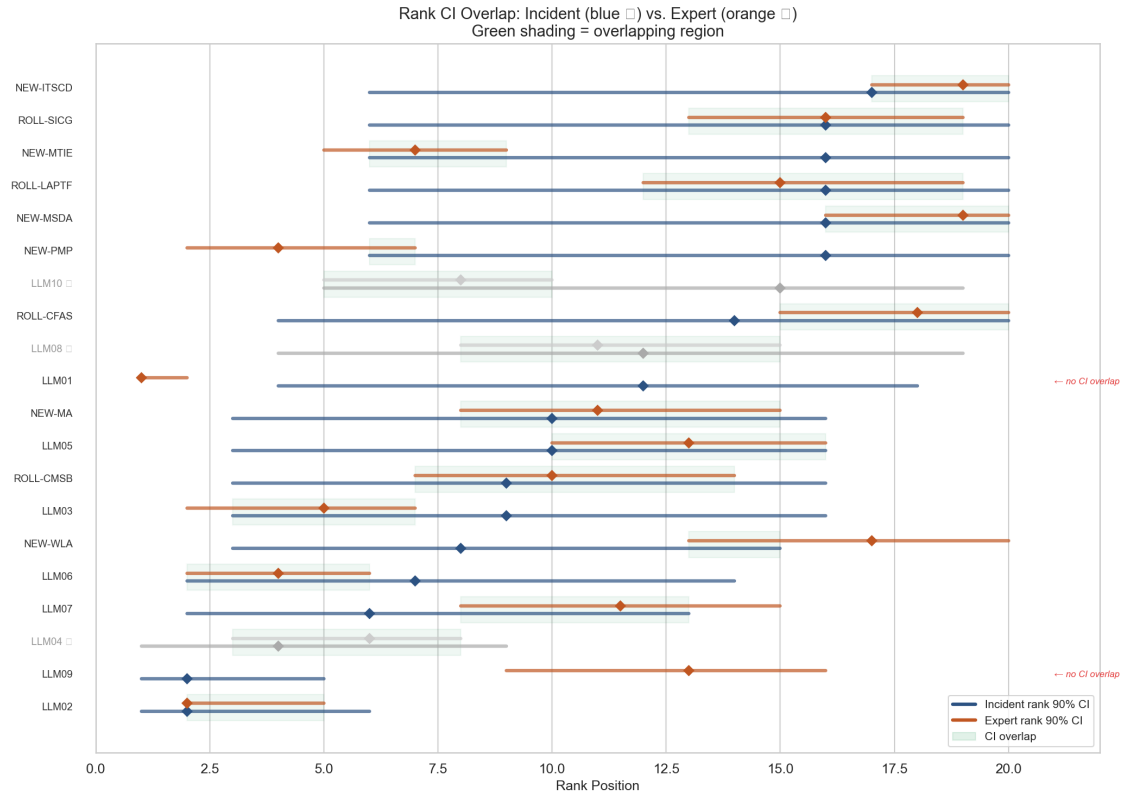
```
    else:
        color = '#e53e3e'

    if is_fb:
        color = '#999999'

    linestyle = '--' if is_fb else '-'
    linewidth = 2.5 if is_flagged else 1.5

    # Slope line from vote rank (left) to lambda rank (right)
    ax.plot([0, 1], [row['vote_rank'], row['lambda_rank']],
            color=color, linewidth=linewidth, linestyle=linestyle, alpha=0.8)

    # Dots at each end
    ax.plot(0, row['vote_rank'], 'o', color=color, markersize=8,
            markeredgecolor='white', markeredgewidth=1)
    ax.plot(1, row['lambda_rank'], 'o', color=color, markersize=8,
            markeredgecolor='white', markeredgewidth=1)

    # Labels
    ax.text(-0.05, row['vote_rank'], eid, ha='right', va='center',
            fontsize=8, fontweight='bold' if is_flagged else 'normal',
            color='#999' if is_fb else '#333')
    ax.text(1.05, row['lambda_rank'], eid, ha='left', va='center',
            fontsize=8, fontweight='bold' if is_flagged else 'normal',
            color='#999' if is_fb else '#333')

ax.set_xlim(-0.3, 1.3)
ax.set_ylim(21, 0)
ax.set_xticks([0, 1])
ax.set_xticklabels(['Expert Rank\n(vote-derived)', 'Incident␣
 ↪Rank\n(lambda-derived)'],
                   fontsize=12)
ax.set_ylabel('Rank Position', fontsize=12)
ax.set_title(f'Expert vs. Incident Rankings\n'
             f'  = {conc["weighted_kappa_median"]:.2f} '
             f'[{conc["weighted_kappa_ci"][0]:.2f},␣
 ↪{conc["weighted_kappa_ci"][1]:.2f}]',
             fontsize=14)

# Legend
legend_elements = [
    mpatches.Patch(color='#38a169', label='Same tier'),
    mpatches.Patch(color='#d69e2e', label='±1 tier'),
    mpatches.Patch(color='#e53e3e', label='±2+ tiers'),
    plt.Line2D([0], [0], color='#999', linestyle='--', label='Frame-blind'),
]
```

```
ax.legend(handles=legend_elements, loc='lower center', ncol=4,
          bbox_to_anchor=(0.5, -0.08), fontsize=10)

plt.subplots_adjust(left=0.18)
plt.tight_layout()
plt.show()
```



[20]:
```
# Show CI overlap between lambda and vote rank CIs for each entry
# Parse vote rank CIs from the comparison report
vote_ci_data = []
for line in lines:
    if line.startswith('|') and not line.startswith('|---') and 'Entry' not in␣
 ↪line:
        parts = [p.strip() for p in line.split('|')[1:-1]]
        if len(parts) >= 6:
            eid = parts[0]
            try:
                # Parse "12.0 (4.0-18.0)" format
                lam_match = re.match(r'([\d.]+)\s*\(([\d.]+)[--]([\d.]+)\)',␣
 ↪parts[1])
                vote_match = re.match(r'([\d.]+)\s*\(([\d.]+)[--]([\d.]+)\)',␣
 ↪parts[2])
                if lam_match and vote_match:
                    vote_ci_data.append({
                        'entry_id': eid,
                        'lam_med': float(lam_match.group(1)),
                        'lam_lo': float(lam_match.group(2)),
                        'lam_hi': float(lam_match.group(3)),
                        'vote_med': float(vote_match.group(1)),
                        'vote_lo': float(vote_match.group(2)),
                        'vote_hi': float(vote_match.group(3)),
                    })
            except (ValueError, AttributeError):
                continue

vci_df = pd.DataFrame(vote_ci_data).sort_values('lam_med')

fig, ax = plt.subplots(figsize=(14, 10))

for i, row in enumerate(vci_df.itertuples()):
    eid = row.entry_id
    is_fb = eid in FRAME_BLIND

    # Check CI overlap
    overlap_lo = max(row.lam_lo, row.vote_lo)
    overlap_hi = min(row.lam_hi, row.vote_hi)
    has_overlap = overlap_lo <= overlap_hi

    # Draw lambda CI (blue)
    ax.plot([row.lam_lo, row.lam_hi], [i - 0.12, i - 0.12],
            color='#2c5282' if not is_fb else '#aaa', linewidth=3,
            alpha=0.7, solid_capstyle='round')
    ax.plot(row.lam_med, i - 0.12, 'D', color='#2c5282' if not is_fb else␣
 ↪'#aaa',
```

```
            markersize=6, zorder=5)

    # Draw vote CI (orange)
    ax.plot([row.vote_lo, row.vote_hi], [i + 0.12, i + 0.12],
            color='#c05621' if not is_fb else '#ccc', linewidth=3,
            alpha=0.7, solid_capstyle='round')
    ax.plot(row.vote_med, i + 0.12, 'D', color='#c05621' if not is_fb else␣
 ↪'#ccc',
            markersize=6, zorder=5)

    # Shade overlap region
    if has_overlap:
        ax.axhspan(i - 0.4, i + 0.4, xmin=overlap_lo / 22,
                   xmax=overlap_hi / 22,
                   alpha=0.08, color='#38a169')

    # Non-overlapping annotation
    if not has_overlap:
        ax.text(21, i, '← no CI overlap', fontsize=8, color='#e53e3e',
                va='center', style='italic')

    # Label
    label = f'{eid}'
    if is_fb:
        label += '  '
    ax.text(-0.5, i, label, ha='right', va='center', fontsize=9,
            color='#999' if is_fb else '#333')

ax.set_yticks([])
ax.set_xlabel('Rank Position', fontsize=12)
ax.set_xlim(0, 22)
ax.set_title('Rank CI Overlap: Incident (blue  ) vs. Expert (orange  )\n'
             'Green shading = overlapping region', fontsize=14)

legend_elements = [
    plt.Line2D([0], [0], color='#2c5282', linewidth=3, label='Incident rank 90%␣
 ↪CI'),
    plt.Line2D([0], [0], color='#c05621', linewidth=3, label='Expert rank 90%␣
 ↪CI'),
    mpatches.Patch(color='#38a169', alpha=0.15, label='CI overlap'),
]
ax.legend(handles=legend_elements, loc='lower right', fontsize=10)

plt.tight_layout()
plt.show()
```



[21]:
```
display(sidebar(
    'Deep dive: How weighted kappa works',
    '<p>Cohen\'s kappa compares observed agreement to expected agreement by␣
↪chance. '
    'Weighted kappa extends this to ordinal data - disagreements by 1 tier are '
    'penalized less than disagreements by 2+ tiers.</p>'
    '<p>Concretely: we divide the 20 entries into rank tiers (top 5, 6-10,␣
↪11-15, '
    '16-20). Each entry gets a tier from the expert ranking and a tier from the␣
↪'
    'incident ranking. Kappa measures how often these tiers match, minus what␣
↪we '
    'would expect from random assignment.</p>'
    '<p>With only <strong>17 measurable entries</strong> (three are frame-blind␣
↪and excluded), '
    'the sample size is small. Small samples produce wide confidence intervals.␣
↪'
    'A kappa of 0.20 on 17 observations could easily be consistent with true␣
↪kappa '
    'values anywhere from -0.16 to 0.57.</p>'
    '<p>For reference: kappa &lt; 0 = worse than chance, 0-0.20 = slight, '
```

```
    '0.21-0.40 = fair, 0.41-0.60 = moderate, &gt; 0.60 = substantial.</p>'
))

sb = DATA['selection_bias']
display(sidebar(
    'Deep dive: Selection bias test',
    f'<p>We tested whether incident rates differ systematically between strata '
    f'using a Kruskal-Wallis test (a non-parametric test for differences across␣
 ↪groups).</p>'
    f'<p>Result: H = {sb["statistic_value"]:.3f}, p = {sb["p_value"]:.3f}, '
    f'severity = {sb["severity"]}.</p>'
    f'<p>A p-value of {sb["p_value"]:.2f} means we cannot reject the null␣
 ↪hypothesis '
    f'that incident rates are similar across strata. In plain language: the␣
 ↪security '
    f'and ai-harm strata do not show statistically different patterns of entry␣
 ↪prevalence. '
    f'This is reassuring - it means our results are not driven by one stratum␣
 ↪dominating.</p>'
))
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

1.8 Act 8: Where Experts and Incidents Disagree

Five entries have notable tier mismatches. For each, we dig into *why* the disagreement exists — what the data shows and what it might mean.

LLM01 (Prompt Injection): Expert #1, incident #12. Prompt injection is the best-understood LLM attack. Deployed systems defend against it actively — input filtering, output sandboxing, system prompt hardening. Fewer incidents reach public databases because defenses often work. Experts rank it #1 because the attack surface is enormous even when defenses hold. The incident data sees fewer successful exploits, so the model ranks it lower.

LLM09 (Misinformation): Incident #2, expert #13. The corpus contains a large volume of deepfake and AI-generated disinformation incidents from the AIAAIC harm database. Experts may rank misinformation lower because "misinformation" as a category overlaps with other entries (NEW-WLA, ROLL-CMSB) and because many of these incidents describe harm *from* AI rather than a vulnerability *in* an LLM. We will examine this overlap in Act 9B.

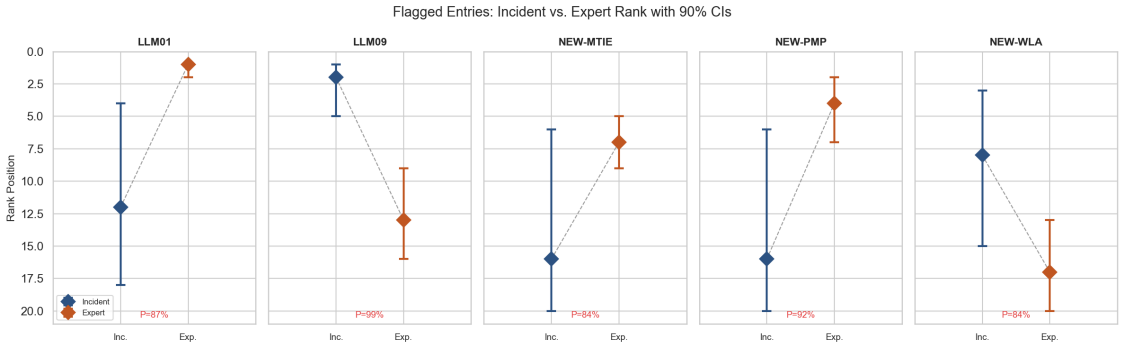
NEW-PMP (Persistent Memory Poisoning) and **NEW-MTIE (MCP Tool Interface Exploitation)**: Expert top-5, almost no incidents yet. These are emerging threats — persistent memory poisoning and MCP tool exploitation are new enough that the public incident record has not caught up. If the goal is to warn practitioners, expert signal may matter more than incident counts for emerging threats.

NEW-WLA (Weaponized LLM Abuse): 863 incidents, expert rank 17. The large incident count is driven by a broad entry definition that captures AI-generated disinformation, deepfake

CSAM, and synthetic media abuse. Experts may rank it low because many of these incidents describe harm *from* AI systems rather than an exploitable vulnerability *in* an LLM.

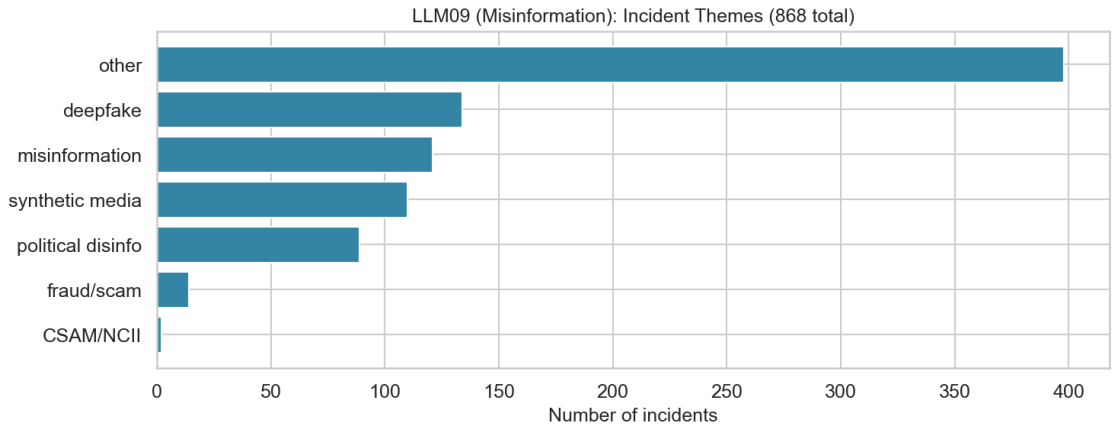
[22]:
```
# Paired dot plots for the 5 flagged entries
flagged_ids = [f['entry_id'] for f in conc['flags']]
flagged_data = comp_df[comp_df['entry_id'].isin(flagged_ids)].copy()

fig, axes = plt.subplots(1, 5, figsize=(16, 5), sharey=True)

for ax, (_, row) in zip(axes, flagged_data.iterrows()):
    eid = row['entry_id']
    # Get CIs from parsed data
    vci_row = vci_df[vci_df['entry_id'] == eid]
    if len(vci_row) == 0:
        continue
    vci_row = vci_row.iloc[0]

    # Lambda rank with CI
    ax.errorbar(0.3, vci_row['lam_med'],
                yerr=[[vci_row['lam_med'] - vci_row['lam_lo']],
                      [vci_row['lam_hi'] - vci_row['lam_med']]],
                fmt='D', color='#2c5282', markersize=10, capsize=5, capthick=2,
                elinewidth=2, label='Incident')

    # Vote rank with CI
    ax.errorbar(0.7, vci_row['vote_med'],
                yerr=[[vci_row['vote_med'] - vci_row['vote_lo']],
                      [vci_row['vote_hi'] - vci_row['vote_med']]],
                fmt='D', color='#c05621', markersize=10, capsize=5, capthick=2,
                elinewidth=2, label='Expert')

    # Connect medians
    ax.plot([0.3, 0.7], [vci_row['lam_med'], vci_row['vote_med']],
            '--', color='#999', linewidth=1)

    ax.set_xlim(-0.1, 1.1)
    ax.set_ylim(21, 0)
    ax.set_xticks([0.3, 0.7])
    ax.set_xticklabels(['Inc.', 'Exp.'], fontsize=9)
    ax.set_title(f'{eid}', fontsize=11, fontweight='bold')

    # Flag probability
    flag = next((f for f in conc['flags'] if f['entry_id'] == eid), None)
    if flag is None:
        continue
    ax.text(0.5, 20.5, f'P={flag["probability"]:.0%}',
            ha='center', fontsize=9, color='#e53e3e')
```

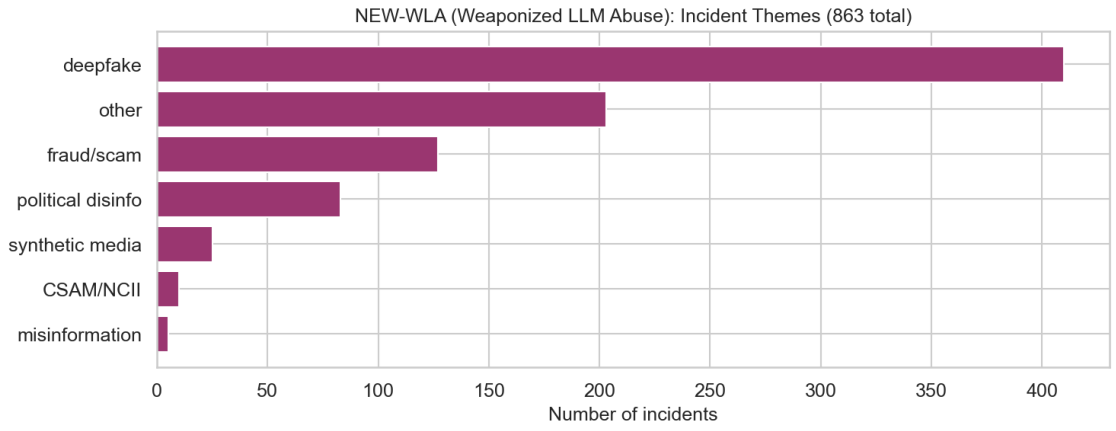
```
axes[0].set_ylabel('Rank Position', fontsize=11)
axes[0].legend(fontsize=8, loc='lower left')

plt.suptitle('Flagged Entries: Incident vs. Expert Rank with 90% CIs',␣
 ↪fontsize=14)
plt.tight_layout()
plt.show()
```



[23]:
```
# Text-mine incident themes for LLM09 and NEW-WLA from prelabels
theme_keywords = {
    'deepfake': ['deepfake', 'deep fake', 'face swap', 'synthetic face'],
    'CSAM/NCII': ['csam', 'child sexual', 'ncii', 'non-consensual intimate'],
    'political disinfo': ['election', 'political', 'propaganda', 'campaign'],
    'fraud/scam': ['fraud', 'scam', 'impersonat'],
    'misinformation': ['misinformation', 'false information', 'fake news',␣
 ↪'hallucin'],
    'synthetic media': ['generated image', 'generated video', 'ai-generated',␣
 ↪'synthetic media'],
    'other': [],
}

def classify_theme(text, keywords_map):
    """Assign first matching theme to avoid double-counting."""
    text_lower = text.lower()
    for theme, keywords in keywords_map.items():
        if theme == 'other':
            continue
        if any(kw in text_lower for kw in keywords):
            return theme
    return 'other'

for target_entry in ['LLM09', 'NEW-WLA']:
```

```
    entry_prelabels = [p for p in DATA['prelabels'] if p['consensus'] ==␣
↪target_entry]
    theme_counts = Counter()
    for p in entry_prelabels:
        theme_counts[classify_theme(p['text'], theme_keywords)] += 1

    fig, ax = plt.subplots(figsize=(10, 4))
    themes_sorted = theme_counts.most_common()
    if themes_sorted:
        ax.barh([t[0] for t in themes_sorted], [t[1] for t in themes_sorted],
                color=ENTRY_COLORS.get(target_entry, '#666'))
        ax.set_xlabel('Number of incidents')
        ax.set_title(f'{target_entry} ({ENTRY_NAMES[target_entry]}): '
                     f'Incident Themes ({len(entry_prelabels)} total)',␣
↪fontsize=12)
        ax.invert_yaxis()
    plt.tight_layout()
    plt.show()
```





1.9 Act 9: What the Data Cannot See

The ranking analysis covers the 17 measurable entries. But two patterns in the data reveal structural limits of what incident-counting can tell us.

1.9.1 9A: "AI Harm Without LLM Vulnerability"

2,394 incidents — about 40% of the corpus — landed in "out of scope." All three models agreed these do not belong to any of the 20 taxonomy entries.

These are real AI harms. Facial recognition that misidentifies people. Algorithmic hiring tools that discriminate. Drones used for surveillance. Recommendation engines that radicalize users. But none of them describe a vulnerability *in* a large language model. They are incidents *from* AI systems, not incidents *of* LLM vulnerabilities.

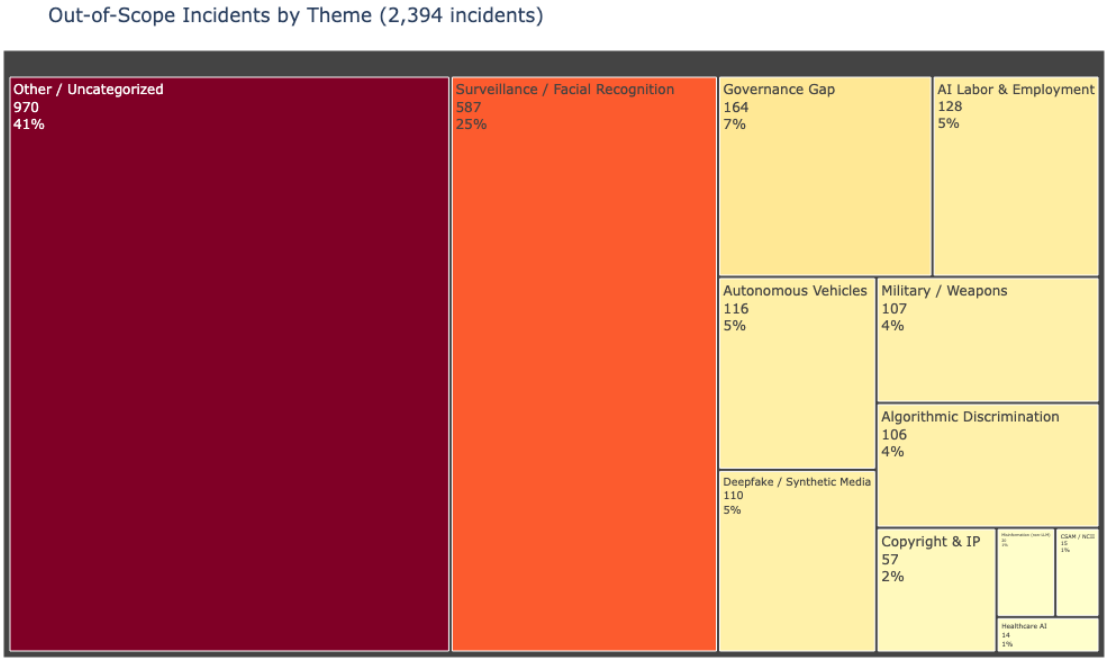
This gap is a feature of the sampling frame, not a failure of the taxonomy. The corpus was built by crawling CVE/GHSA/OSV databases with AI-related keywords. Those keywords pull in any incident that mentions "AI" or "machine learning," regardless of whether an LLM is involved. The AIAAIC harm database, by design, covers all AI-related harms.

The out-of-scope cluster matters because it shows the boundary of what this methodology can measure. Incident-counting works when incidents map to taxonomy entries. For harms that sit outside the taxonomy — because they involve non-LLM AI, or because they describe societal effects rather than technical vulnerabilities — the incident signal is silent.

[24]:
```
from IPython.display import Image as _StaticImage
# Classify OOS incidents into thematic clusters
oos_prelabels = [p for p in DATA['prelabels'] if p['consensus'] ==␣
 ↪'out-of-scope']

oos_theme_keywords = {
    'Surveillance / Facial Recognition': ['surveillance', 'facial recognition',␣
 ↪'face recognition',
                                          'biometric', 'monitoring',␣
 ↪'tracking'],
    'Algorithmic Discrimination': ['discrimination', 'bias', 'discriminat',␣
 ↪'racial',
                                   'gender bias', 'hiring'],
    'Deepfake / Synthetic Media': ['deepfake', 'deep fake', 'synthetic media',␣
 ↪'face swap',
                                   'generated image', 'generated video'],
    'Autonomous Vehicles': ['self-driving', 'autonomous vehicle', 'autopilot',␣
 ↪'tesla crash',
                            'self driving'],
    'AI Labor & Employment': ['worker', 'labor', 'employment', 'gig economy',␣
 ↪'automation',
```

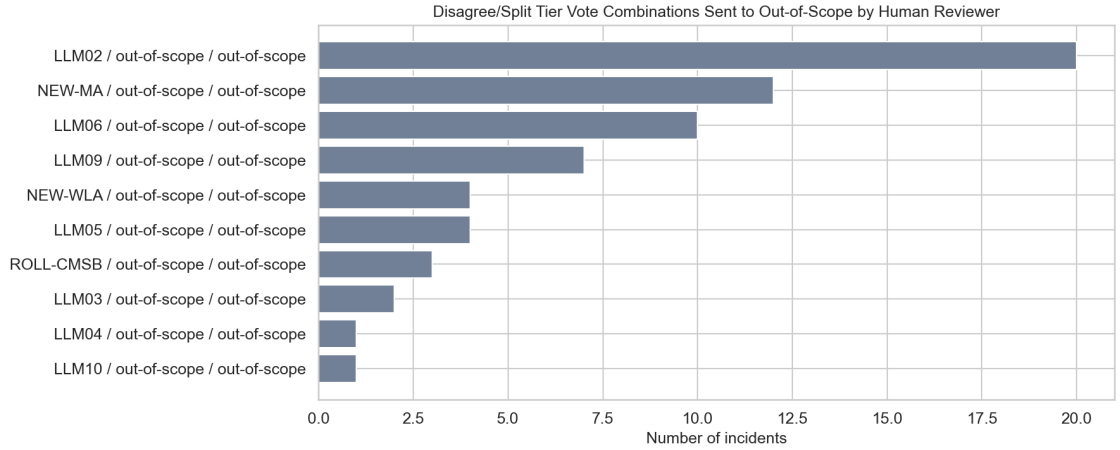
```
                                'job loss', 'replaced by ai'],
    'Copyright & IP': ['copyright', 'intellectual property', 'plagiarism',␣
 ↪'training data',
                       'scraped'],
    'CSAM / NCII': ['csam', 'child sexual', 'ncii', 'non-consensual intimate',
                    'child abuse material'],
    'Governance Gap': ['regulation', 'governance', 'accountability',␣
 ↪'oversight',
                       'unregulated', 'policy'],
    'Misinformation (non-LLM)': ['misinformation', 'disinformation', 'fake␣
 ↪news',
                                 'propaganda', 'conspiracy'],
    'Healthcare AI': ['healthcare', 'medical', 'diagnosis', 'patient',␣
 ↪'clinical'],
    'Military / Weapons': ['military', 'weapon', 'drone strike', 'lethal␣
 ↪autonomous',
                           'warfare'],
}

oos_theme_counts = Counter()
for p in oos_prelabels:
    text_lower = p['text'].lower()
    matched = False
    for theme, keywords in oos_theme_keywords.items():
        if any(kw in text_lower for kw in keywords):
            oos_theme_counts[theme] += 1
            matched = True
            break  # first match wins to avoid double-counting
    if not matched:
        oos_theme_counts['Other / Uncategorized'] += 1

# Build treemap
themes = list(oos_theme_counts.keys())
counts = list(oos_theme_counts.values())

fig = px.treemap(
    names=themes,
    parents=['' for _ in themes],
    values=counts,
    title=f'Out-of-Scope Incidents by Theme ({len(oos_prelabels):,} incidents)',
    color=counts,
    color_continuous_scale='YlOrRd',
)
fig.update_traces(
    textinfo='label+value+percent root',
    textfont_size=12,
)
```

```
fig.update_layout(
    height=500,
    margin=dict(t=50, l=10, r=10, b=10),
    coloraxis_showscale=False,
)
# Render as static PNG only - preserves PDF compatibility and avoids the
# nbconvert 'html-only mimetype' warning on plotly interactive HTML.
display(_StaticImage(fig.to_image(format='png', width=1000, height=600)))
```



[25]:
```
# What the human reviewer overrode to out-of-scope in disagree/split tiers
disagree_to_oos = [g for g in DATA['goldset']
                   if g['adjudicated'] == 'accept'
                   and g['llm_consensus'] == 'out-of-scope'
                   and g['incident_id'] in {p['incident_id'] for p in␣
 ↪DATA['prelabels']
                                            if p['triage_tier'] in␣
 ↪('disagree', 'split')}]

# For disagree-tier incidents that went to OOS, look at what the models␣
 ↪originally voted
disagree_oos_votes = []
prelabel_lookup = {p['incident_id']: p for p in DATA['prelabels']}
for g in DATA['goldset']:
    if (g['llm_consensus'] == 'out-of-scope'
        and g['adjudicated'] == 'accept'
```

```
        and g['incident_id'] in prelabel_lookup):
        pl = prelabel_lookup[g['incident_id']]
        if pl['triage_tier'] in ('disagree', 'split'):
            votes = tuple(sorted(v['entry_id'] for v in pl['model_votes']))
            disagree_oos_votes.append(votes)

# Count the top vote triples/pairs that ended up OOS
vote_combos = Counter(disagree_oos_votes)
top_combos = vote_combos.most_common(10)

if top_combos:
    fig, ax = plt.subplots(figsize=(12, 5))
    labels = [' / '.join(c[0]) for c in top_combos]
    values = [c[1] for c in top_combos]
    ax.barh(labels, values, color='#718096')
    ax.set_xlabel('Number of incidents')
    ax.set_title('Disagree/Split Tier Vote Combinations Sent to Out-of-Scope by␣
 ↪Human Reviewer',
                 fontsize=12)
    ax.invert_yaxis()
    plt.tight_layout()
    plt.show()
else:
    print("No disagree/split-tier incidents were adjudicated to out-of-scope.")
```



1.9.2 9B: The LLM09 / NEW-WLA / ROLL-CMSB Confusion Boundary

A **confusion boundary** is a region where categories overlap enough that classifiers — and sometimes humans — cannot reliably tell them apart. The problem is not that the classifier is broken. The problem is that the categories share real conceptual territory.

The data shows a clear confusion boundary between three entries:

- **LLM09 (Misinformation)**: the output is false or misleading
- **NEW-WLA (Weaponized LLM Abuse)**: an adversary uses AI as a weapon
- **ROLL-CMSB (Cross-Modal Safety Bypass)**: the attack uses image/video/audio modalities

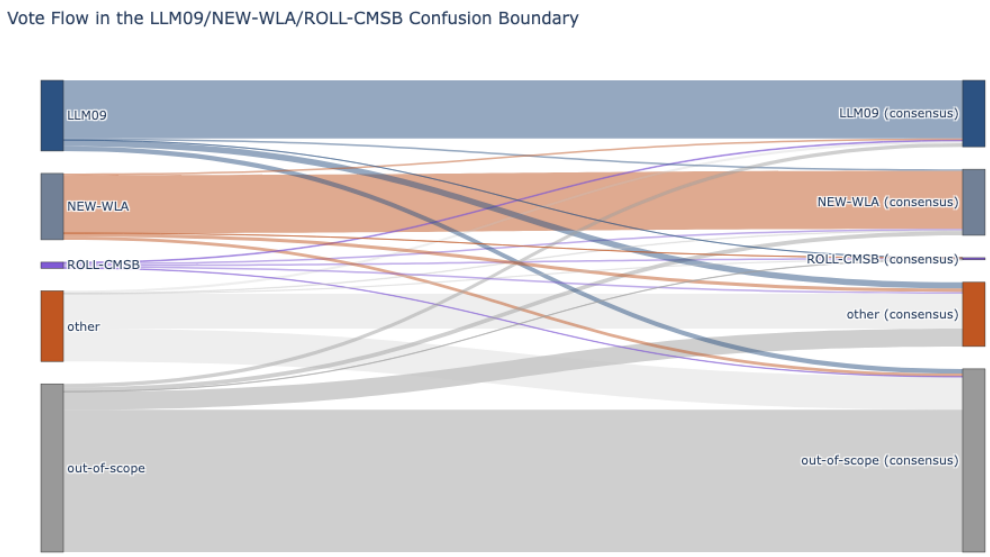
Consider a deepfake video that spreads political disinformation. Which entry does it belong to? It is misleading content (LLM09). It was created using AI as a weapon (NEW-WLA). It exploits an image/video generation modality (ROLL-CMSB). The three categories overlap in real-world incidents, and the overlap is not a classification error — it reflects genuine ambiguity in the taxonomy.

This matters for interpretation. When the incident data ranks LLM09 at #2, some of that signal comes from incidents that could equally have been classified as NEW-WLA or ROLL-CMSB. The confusion boundary inflates counts for whichever entry the classifier happens to prefer and deflates counts for the others. The Bayesian model corrects for measured precision (how often each entry's classifications are right), but it cannot correct for ambiguity that the gold-set reviewers themselves found difficult to resolve.

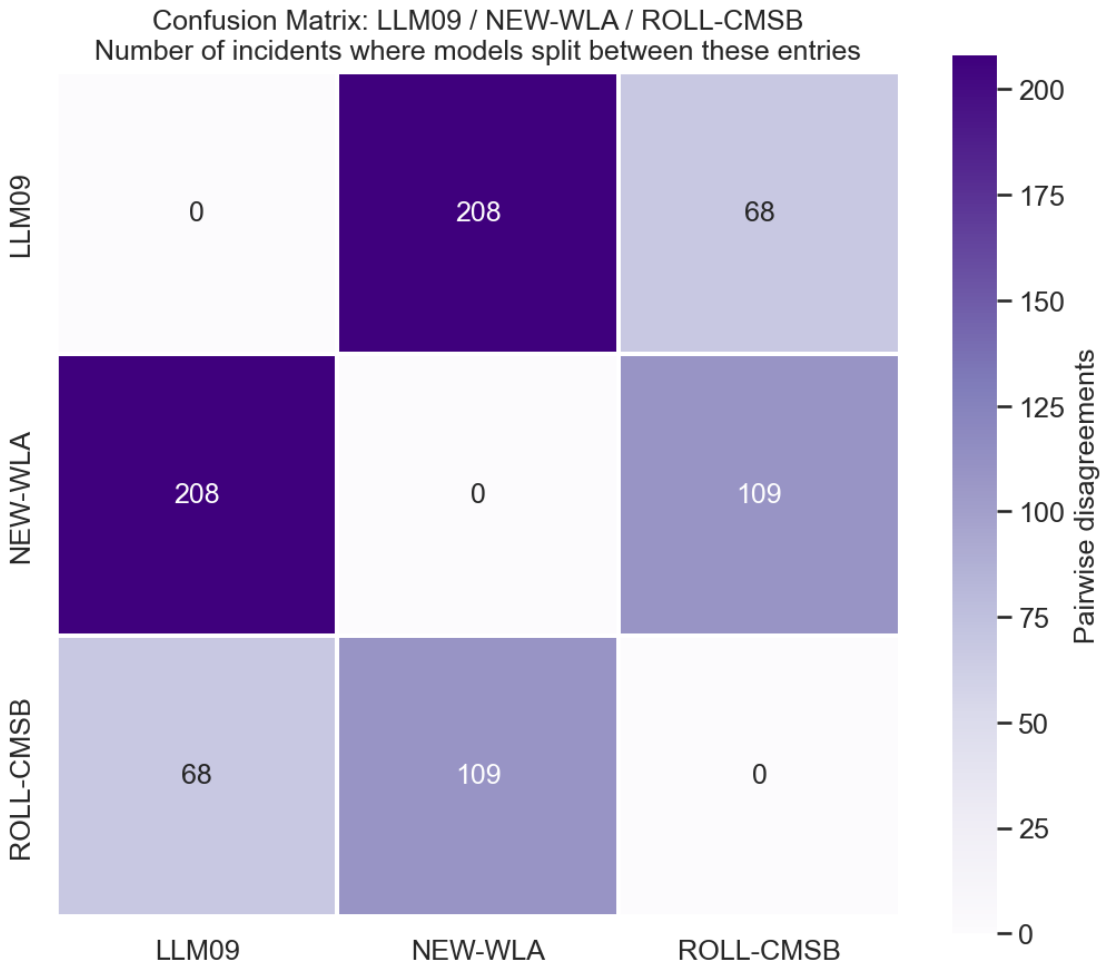
[26]:
```
from IPython.display import Image as _StaticImage
# Build Sankey diagram: model votes → consensus for the confusion boundary␣
 ↪entries
boundary_entries = {'LLM09', 'NEW-WLA', 'ROLL-CMSB', 'out-of-scope'}

# Find incidents where any model voted for a boundary entry
boundary_incidents = []
for p in DATA['prelabels']:
    votes = [v['entry_id'] for v in p['model_votes']]
    if any(v in boundary_entries for v in votes):
        boundary_incidents.append(p)

# Count flows: model vote → consensus
model_names = ['Qwen', 'Llama', 'DeepSeek']
flows = defaultdict(int)

for p in boundary_incidents:
    consensus = p['consensus'] if p['consensus'] in boundary_entries else␣
 ↪'other'
    for v in p['model_votes']:
        vote = v['entry_id'] if v['entry_id'] in boundary_entries else 'other'
        model_short = v['model_id'].split('/')[-1].split('-')[0]
        flows[(vote, consensus)] += 1

# Build Sankey nodes and links
source_labels = sorted(boundary_entries | {'other'})
target_labels = [f'{s} (consensus)' for s in sorted(boundary_entries |␣
 ↪{'other'})]
all_labels = source_labels + target_labels

source_idx = {s: i for i, s in enumerate(source_labels)}
```

```
target_idx = {s: i + len(source_labels) for i, s in␣
 ↪enumerate(sorted(boundary_entries | {'other'}))}

sankey_source = []
sankey_target = []
sankey_value = []
sankey_color = []

entry_sankey_colors = {
    'LLM09': 'rgba(44, 82, 130, 0.5)',
    'NEW-WLA': 'rgba(192, 86, 33, 0.5)',
    'ROLL-CMSB': 'rgba(128, 90, 213, 0.5)',
    'out-of-scope': 'rgba(160, 160, 160, 0.5)',
    'other': 'rgba(200, 200, 200, 0.3)',
}

for (vote, consensus), count in flows.items():
    if count < 3:
        continue
    sankey_source.append(source_idx[vote])
    t_key = consensus
    sankey_target.append(target_idx[t_key])
    sankey_value.append(count)
    sankey_color.append(entry_sankey_colors.get(vote, 'rgba(200,200,200,0.3)'))

fig = go.Figure(go.Sankey(
    node=dict(
        pad=20,
        thickness=20,
        line=dict(color='#333', width=0.5),
        label=all_labels,
        color=['#2c5282', '#718096', '#805ad5', '#c05621', '#999'] * 2,
    ),
    link=dict(
        source=sankey_source,
        target=sankey_target,
        value=sankey_value,
        color=sankey_color,
    )
))
fig.update_layout(
    title='Vote Flow in the LLM09/NEW-WLA/ROLL-CMSB Confusion Boundary',
    font_size=11,
    height=500,
)
# Render as static PNG only - preserves PDF compatibility and avoids the
# nbconvert 'html-only mimetype' warning on plotly interactive HTML.
```

```
display(_StaticImage(fig.to_image(format='png', width=1000, height=600)))
```



[27]:
```
# 3x3 confusion matrix: how often each model pair disagrees within the boundary
boundary_list = ['LLM09', 'NEW-WLA', 'ROLL-CMSB']
pair_confusion = pd.DataFrame(0, index=boundary_list, columns=boundary_list)

for p in DATA['prelabels']:
    if p['triage_tier'] in ('split', 'disagree'):
        votes = [v['entry_id'] for v in p['model_votes']]
        boundary_votes = [v for v in votes if v in boundary_list]
        unique_bv = list(set(boundary_votes))
        for i in range(len(unique_bv)):
            for j in range(i + 1, len(unique_bv)):
                pair_confusion.loc[unique_bv[i], unique_bv[j]] += 1
                pair_confusion.loc[unique_bv[j], unique_bv[i]] += 1

fig, ax = plt.subplots(figsize=(7, 6))
sns.heatmap(pair_confusion, annot=True, fmt='d', cmap='Purples',
            linewidths=1, ax=ax, square=True,
            cbar_kws={'label': 'Pairwise disagreements'})
ax.set_title('Confusion Matrix: LLM09 / NEW-WLA / ROLL-CMSB\n'
             'Number of incidents where models split between these entries',
             fontsize=12)
plt.tight_layout()
plt.show()
```



[28]:
```
# Show 2-3 real incidents from the confusion boundary
boundary_examples = []
for p in DATA['prelabels']:
    if p['triage_tier'] == 'disagree':
        votes = set(v['entry_id'] for v in p['model_votes'])
        if votes & {'LLM09', 'NEW-WLA', 'ROLL-CMSB'} and len(votes &␣
 ↪set(boundary_list)) >= 2:
            boundary_examples.append(p)
    if len(boundary_examples) >= 3:
        break

# Fall back to split tier if not enough disagree examples
if len(boundary_examples) < 2:
    for p in DATA['prelabels']:
        if p['triage_tier'] == 'split':
            votes = set(v['entry_id'] for v in p['model_votes'])
```

```
            if len(votes & set(boundary_list)) >= 2:
                boundary_examples.append(p)
        if len(boundary_examples) >= 3:
            break

display(HTML('<h4>Real incidents from the confusion boundary</h4>'))
for ex in boundary_examples[:3]:
    text = html.escape(shorten(ex['text'], width=300, placeholder='...'))
    vote_html = ''.join(
        f'<li><strong>{html.escape(v["model_id"].split("/")[-1])}</strong>: '
        f'{html.escape(v["entry_id"])} ({v["confidence"]:.0%})</li>'
        for v in ex['model_votes']
    )

    # Check if this incident was adjudicated
    adj = next((g for g in DATA['goldset'] if g['incident_id'] ==␣
↪ex['incident_id']), None)
    adj_html = ''
    if adj:
        labels_str = html.escape(", ".join(adj["labels"]) if adj["labels"] else␣
↪"out-of-scope")
        notes_str = html.escape(adj["notes"]) if adj.get("notes") else ""
        adj_html = (f'<p><strong>Human decision:</strong> {html.
↪escape(adj["adjudicated"])} → '
                    f'{labels_str}'
                    f'{" - " + notes_str if notes_str else ""}</p>')

    display(HTML(
        f'<div style="border: 1px solid #805ad5; border-radius: 8px; padding:␣
↪1em; '
        f'margin: 0.5em 0; background: #faf5ff;">'
        f'<code>{html.escape(ex["incident_id"])}</code> · tier: '
        f'{html.escape(ex["triage_tier"])}<br>'
        f'<p style="color: #333; font-style: italic;">{text}</p>'
        f'<ul style="margin: 0.5em 0;">{vote_html}</ul>'
        f'{adj_html}'
        f'</div>'
    ))
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

1.10 Act 10: What This Means

Where the data and experts agree. LLM02 (Sensitive Information Disclosure) sits near the top by both measures — experts rank it #2 and incidents rank it #2. ROLL-SICG, NEW-ITSCD, and NEW-MSDA are consistently near the bottom by both signals. These positions are stable across the uncertainty ranges.

Where the data pushes back. LLM09's incident volume is much higher than its expert rank. Part of this comes from the broad entry definition, which captures AI-adjacent harms (deepfake misuse, synthetic disinformation) that may not represent LLM vulnerabilities in the narrow sense. NEW-WLA shows a similar pattern. The confusion boundary between LLM09, NEW-WLA, and ROLL-CMSB inflates whichever entry the classifier prefers and makes all three counts less reliable than entries with cleaner boundaries.

What the experts see that incidents miss. NEW-PMP and NEW-MTIE have almost no incidents in the public record but strong expert signal. These are forward-looking entries — persistent memory poisoning and MCP tool exploitation are new enough that the incident databases have not caught up. If the purpose of the Top 10 is to warn practitioners about threats they will face, expert signal matters more than incident counts for emerging threats.

What this methodology can and cannot do. This is a triangulation tool. It checks one signal (expert surveys) against another (incident data). Neither signal is the truth. The incident data has known structural biases: the sampling frame misses incidents that are not publicly reported, the classifier has measured error rates, and the taxonomy-frame circularity (F-circ) means we are partially measuring the classifier's preferences rather than the true threat distribution. The expert data has its own biases: availability bias, recency effects, anchoring to prior Top 10 lists.

The value is in the comparison, not in either signal alone. Where experts and incidents agree, confidence is higher. Where they diverge, the disagreement itself is the finding — it points to entries where one signal or the other may be systematically distorted.

The kappa ceiling is structural. Some of the disagreement between expert and incident rankings is informative — it reveals real differences between "what experts worry about" and "what has actually happened." Perfect agreement would be surprising and arguably suspicious. The current kappa of 0.20 [−0.16, 0.57] is consistent with weak-to-moderate agreement, but the confidence interval is too wide to draw firm conclusions. A larger corpus, better-defined entry boundaries, and independent recall measurement would all narrow the interval and sharpen the comparison.

Accepted limitation: ai-harm precision. The 323 precision verifications were drawn entirely from the security stratum. The ai-harm stratum (92 in-scope incidents across 8 entry assignments, of which only 3 received recall posteriors with material evidence — LLM09, LLM04, NEW-MA; NEW-WLA has only 1 observation above the pure prior) has no direct precision measurements — ai-harm precision keys are absent from the calibration data entirely. The model falls back to a flat Beta(1,1) = Uniform(0,1) prior for ai-harm precision, meaning it assumes no prior knowledge about how precise the classifier is on ai-harm incidents (prior mean 0.5). Closing this gap would require sourcing additional ai-harm incidents beyond the existing corpus, which is outside this project's scope. The disclosure in Acts 4 and 5 describes how the model handles missing precision data.